



Network Management and Administration (IE2110)
2nd Year, 2nd Semester

Assignment

Assignment 01 – Zimbra

Submitted to
Sri Lanka Institute of Information Technology

IT24103506

Siriwardana S.A.D.V.I.

In partial fulfillment of the requirements for the
Bachelor of Science Special Honors Degree in Information Technology

April 12, 2026

Declaration

I certify that this report does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text.

Table of Contents:

1. [Part A – Initial Setup](#)
2. [Part B – DHCP Server](#)
3. [Part C – Zimbra Mail Server](#)
4. [Part D – Samba Active Directory Domain Controller](#)
5. [Part E – Web Server](#)
6. [Final System Verification](#)

1. Part A – Initial Setup

1.1 Virtual Machine Creation:

Four virtual machines were created using Oracle VirtualBox to simulate a network environment.

- One virtual machine was configured as a CentOS Server.
- One virtual machine was configured as an Ubuntu Server.
- Two machines were configured as Ubuntu Clients.

These machines will be used to deploy and test network services.

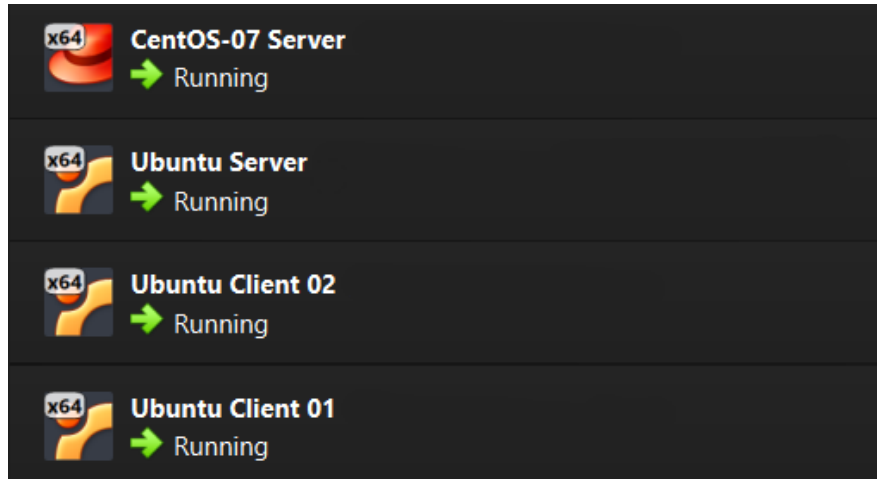


Figure 1.1: Virtual Machines Created in VirtualBox

Description:

The figure illustrates the four virtual machines created in Oracle VirtualBox for the network setup. An additional Ubuntu Server was introduced to deploy specific services due to compatibility limitations in CentOS 7. This approach ensured better stability, easier configuration, and successful implementation of all required network services.

1.2 Operating System Installation

The required operating systems were installed on each virtual machine.

- CentOS 7 Minimal was installed on the server machine.
- Ubuntu 22.04 LTS was installed on both client machines.
- Ubuntu 22.04 LTS was installed as an Ubuntu Server machine for parts D and E

All systems were successfully installed and booted without errors.

```
CentOS Linux 7 (Core)
Kernel 3.10.0-327.36.1.el7.x86_64 on an x86_64

localhost login: vsiriwardana
Password:
Last login: Thu Apr  2 15:04:58 on tty1
[vsiriwardana@localhost ~]$ _
```

Figure 1.2: CentOS Server Operating System Interface

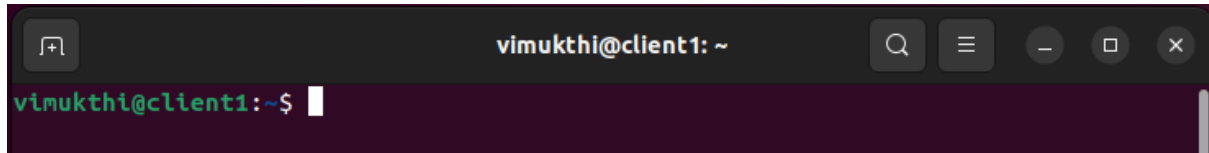
A terminal window titled 'vimukthi@client1: ~' with search, menu, and window control icons in the title bar. The terminal shows the prompt 'vimukthi@client1:~\$' with a cursor.

Figure 1.3: Ubuntu Client 1 Operating System

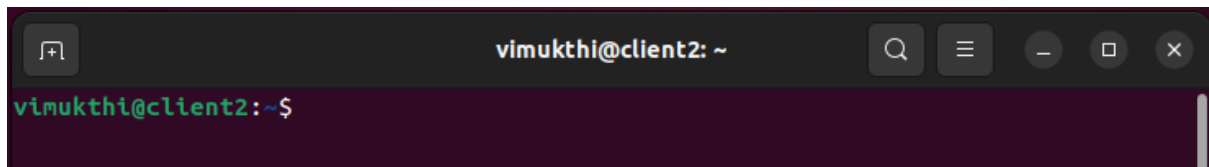
A terminal window titled 'vimukthi@client2: ~' with search, menu, and window control icons in the title bar. The terminal shows the prompt 'vimukthi@client2:~\$' with a cursor.

Figure 1.4: Ubuntu Client 2 Operating System

Description:

These figures illustrate the successful installation and running state of the operating system on the server and client virtual machines.

1.3 Network Configuration

To enable communication between virtual machines, a network configuration was established using Oracle VirtualBox. All machines were connected to the same internal network to allow direct communication between the server and client systems.

The network adapter settings were configured as follows:

- All VirtualBox machines were connected using Internal Network.
- The Network name was set to “NMA-NET”.
- This configuration ensures that all machines are within the same isolated network environment.

1.3.1 VirtualBox Network Settings

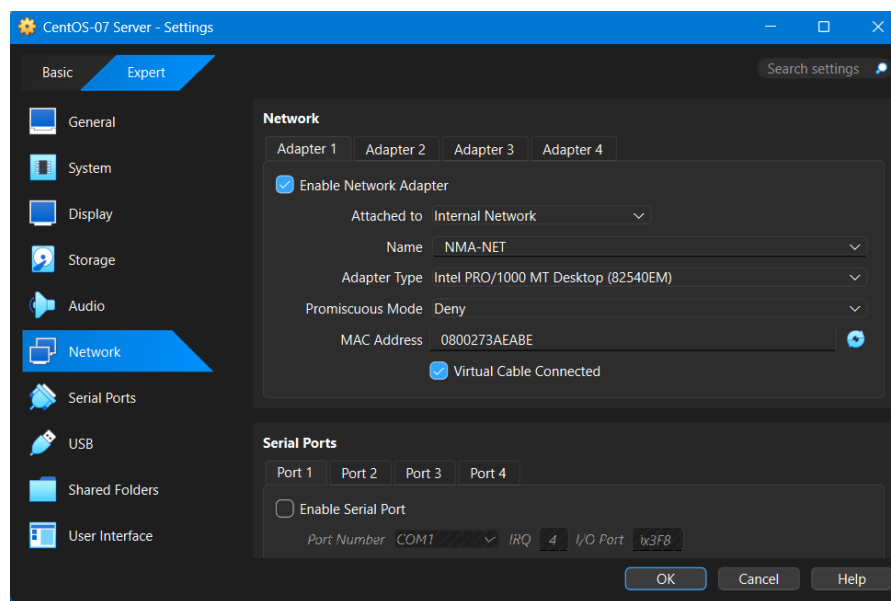


Figure 1.5: Network Configuration of CentOS Server

Description:

This screenshot confirms the network settings of the CentOS server configured in Oracle VirtualBox. The adapter is connected to an Internal Network named “NMA-NET” to allow communication between virtual machines.

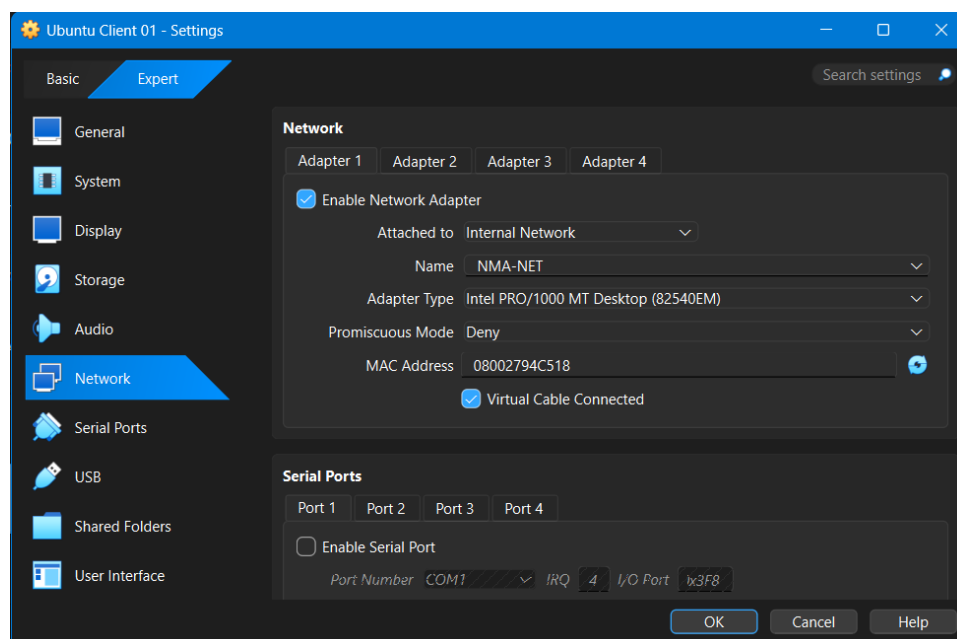


Figure 1.6: Network Configuration of Ubuntu Clients

Description:

The figure illustrates the network configuration of the Ubuntu client machines, where both are connected to the same Internal Network “NMA-NET” to ensure connectivity within the virtual network.

1.4 IP Address Configuration

To enable communication between the server and client machines, static IP addresses were assigned to each virtual machine within the same network range.

The network was configured using the IP range 192.168.6.0/24, ensuring that all devices belong to the same subnet.

The IP addresses assigned are as follows:

- CentOS Server: 192.168.6.1
- Ubuntu Client 1: 192.168.6.10
- Ubuntu Client 2: 192.168.6.11

The IP configuration was performed using appropriate network configuration tools in each operating system.

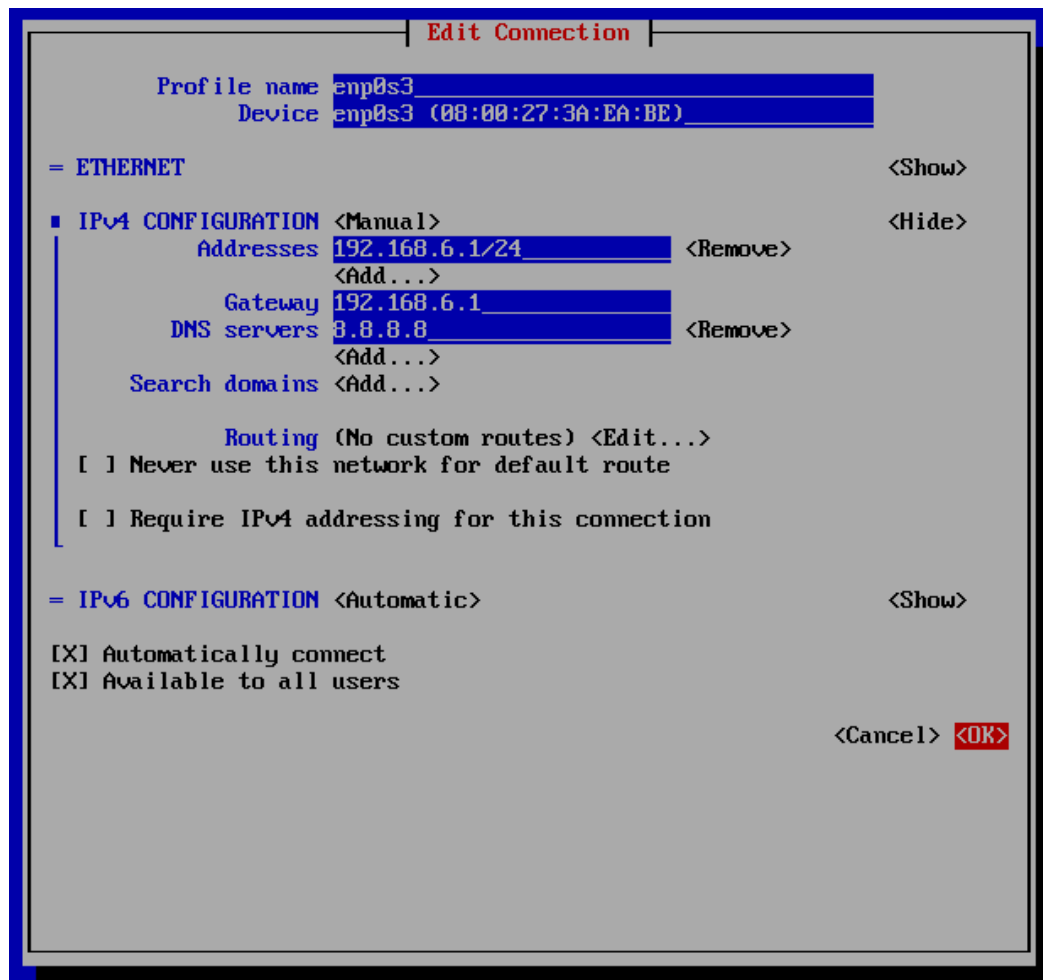


Figure 1.7: IP configuration of CentOS Server

Description:

This screenshot illustrates the network interface configuration on the CentOS server using the *nmtui* tool. The internal network interface was assigned a static IP address (192.168.6.1/24) and configured with a DNS server, enabling communication within the network.

```

root@localhost vsiriwardanal# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 08:00:27:3a:ea:be brd ff:ff:ff:ff:ff:ff
    inet 192.168.6.1/24 brd 192.168.6.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe3a:eabe/64 scope link
        valid_lft forever preferred_lft forever
root@localhost vsiriwardanal#

```

Figure 1.8: Static IP Configuration of CentOS Server

Description:

This result confirms the network interface configuration on the CentOS server, with a static IP address (192.168.6.1/24) assigned. This ensures that the server can act as a central node for network services.

```

vimukthi@client1: ~
GNU nano 6.2 /etc/netplan/01-netcfg.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.6.10/24
      gateway4: 192.168.6.1
      nameservers:
        addresses: [8.8.8.8]

```

Figure 1.9: Netplan Configuration for Ubuntu Client 1

Description:

This figure illustrates the Netplan configuration file used to assign a static IP address (192.168.6.10/24) to Ubuntu Client 1, including gateway and DNS settings.

```

vimukthi@client1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:94:c5:18 brd ff:ff:ff:ff:ff:ff
    inet 192.168.6.10/24 brd 192.168.6.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe94:c518/64 scope link
        valid_lft forever preferred_lft forever
vimukthi@client1:~$

```

Figure 1.10: IP Address Verification of Ubuntu Client 1

Description:

This screenshot displays the output of the *ip a* command on Ubuntu Client 1, confirming that the static IP address has been successfully assigned.

```

vimukthi@client2: ~
GNU nano 6.2 /etc/netplan/01-netcfg.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.6.11/24
      gateway4: 192.168.6.1
      nameservers:
        addresses: [8.8.8.8]

```

Figure 1.11: Netplan Configuration for Ubuntu Client 2

Description:

This screenshot confirms the Netplan configuration file used to assign a static IP address (192.168.6.11/24) to Ubuntu Client 2, ensuring it is within the same network.

```

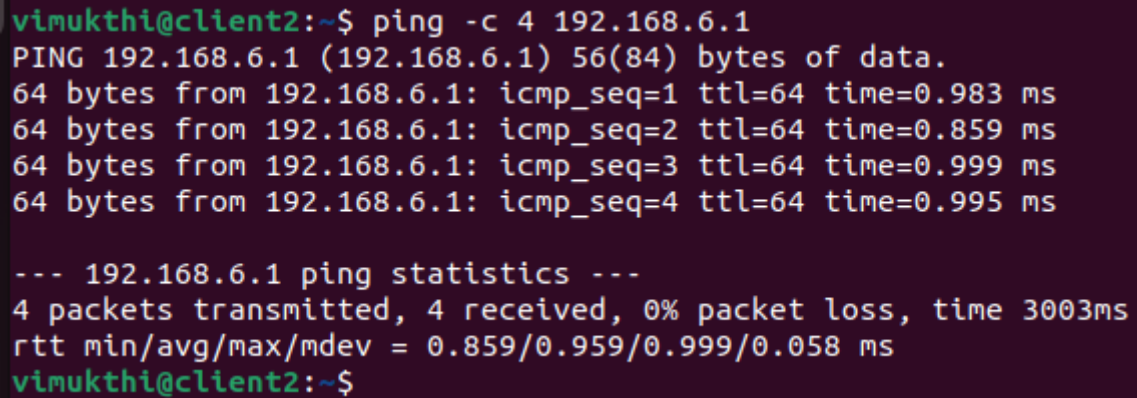
vimukthi@client2:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:13:92:6e brd ff:ff:ff:ff:ff:ff
    inet 192.168.6.11/24 brd 192.168.6.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe13:926e/64 scope link
        valid_lft forever preferred_lft forever
vimukthi@client2:~$

```

Figure 1.12: IP address Verification of Ubuntu Client 2

This screenshot displays the output of the *ip a* command on Ubuntu Client 2, confirming the correct assignment of the static IP address.

1.5 Network Connectivity Verification



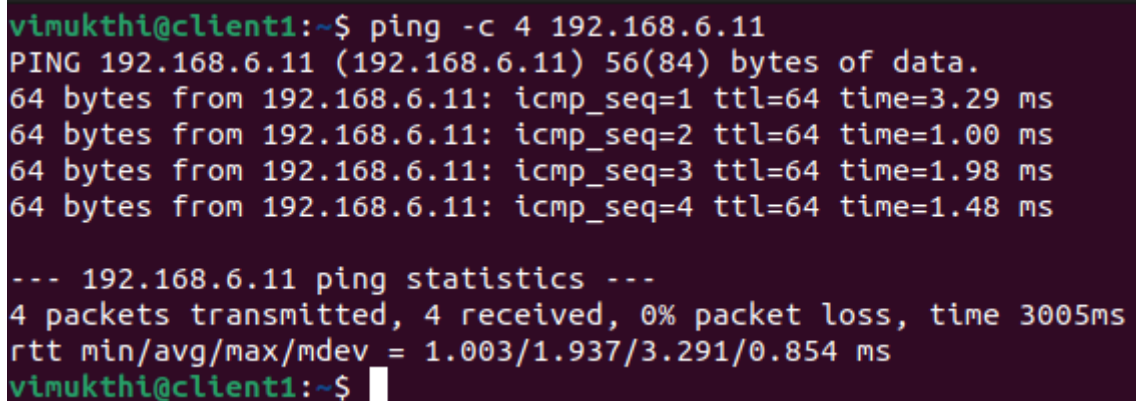
```
vimukthi@client2:~$ ping -c 4 192.168.6.1
PING 192.168.6.1 (192.168.6.1) 56(84) bytes of data.
64 bytes from 192.168.6.1: icmp_seq=1 ttl=64 time=0.983 ms
64 bytes from 192.168.6.1: icmp_seq=2 ttl=64 time=0.859 ms
64 bytes from 192.168.6.1: icmp_seq=3 ttl=64 time=0.999 ms
64 bytes from 192.168.6.1: icmp_seq=4 ttl=64 time=0.995 ms

--- 192.168.6.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 0.859/0.959/0.999/0.058 ms
vimukthi@client2:~$
```

Figure 1.13: Connectivity Test from Client2 to Server

Description:

This screenshot demonstrates successful communication from Ubuntu Client 2 to the CentOS server using the ping command, verifying that the network is correctly configured.



```
vimukthi@client1:~$ ping -c 4 192.168.6.11
PING 192.168.6.11 (192.168.6.11) 56(84) bytes of data.
64 bytes from 192.168.6.11: icmp_seq=1 ttl=64 time=3.29 ms
64 bytes from 192.168.6.11: icmp_seq=2 ttl=64 time=1.00 ms
64 bytes from 192.168.6.11: icmp_seq=3 ttl=64 time=1.98 ms
64 bytes from 192.168.6.11: icmp_seq=4 ttl=64 time=1.48 ms

--- 192.168.6.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.003/1.937/3.291/0.854 ms
vimukthi@client1:~$
```

Figure 1.14: Connectivity Test from Client1 to Client2

Description:

This figure demonstrates successful communication from Ubuntu Client 1 to Client 2 using the ping command, verifying that the network is correctly configured.

```

[root@localhost ~]# ping -c 4 192.168.6.11
PING 192.168.6.11 (192.168.6.11) 56(84) bytes of data.
64 bytes from 192.168.6.11: icmp_seq=1 ttl=64 time=1.32 ms
64 bytes from 192.168.6.11: icmp_seq=2 ttl=64 time=1.99 ms
64 bytes from 192.168.6.11: icmp_seq=3 ttl=64 time=0.747 ms
64 bytes from 192.168.6.11: icmp_seq=4 ttl=64 time=1.29 ms

--- 192.168.6.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3008ms
rtt min/avg/max/mdev = 0.747/1.341/1.993/0.443 ms
[root@localhost ~]#

```

Figure 1.15: Connectivity Test from Server to Client 2

Description:

The above output confirms successful communication from the server to Client 2 using the ping command, verifying proper network connectivity.

```

[root@localhost ~]# ping -c 4 192.168.6.10
PING 192.168.6.10 (192.168.6.10) 56(84) bytes of data.
64 bytes from 192.168.6.10: icmp_seq=1 ttl=64 time=1.73 ms
64 bytes from 192.168.6.10: icmp_seq=2 ttl=64 time=1.55 ms
64 bytes from 192.168.6.10: icmp_seq=3 ttl=64 time=0.530 ms
64 bytes from 192.168.6.10: icmp_seq=4 ttl=64 time=0.523 ms

--- 192.168.6.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3008ms
rtt min/avg/max/mdev = 0.523/1.084/1.734/0.563 ms
[root@localhost ~]#

```

Figure 1.16: Connectivity Test from Server to Client 1

Description:

This screenshot illustrates successful communication from the server to Client 1 using the ping command, confirming that all devices can communicate within the network.

2. Part B – DHCP Server

The DHCP server package was installed on the CentOS server using the yum package manager to enable automatic IP address allocation.

2.1 DHCP Server Installation

```
Installed:
  dhcp.x86_64 12:4.2.5-83.el7.centos.2

Dependency Installed:
  bind-export-libs.x86_64 32:9.11.4-26.P2.el7_9.16

Dependency Updated:
  dhclient.x86_64 12:4.2.5-83.el7.centos.2      dhcp-common.x86_64 12:4.2.5-83.el7.centos.2
  dhcp-libs.x86_64 12:4.2.5-83.el7.centos.2

Complete!
[root@localhost ~]#
```

Figure 2.1: DHCP Server Installation on CentOS

Description:

This figure illustrates the successful installation of the DHCP server package using the *yum install dhcp* command, including all required dependencies.

2.2 DHCP Server Configuration

```
GNU nano 2.3.1      File: /etc/dhcp/dhcpd.conf

subnet 192.168.6.0 netmask 255.255.255.0 {
    range 192.168.6.100 192.168.6.200;
    option routers 192.168.6.1;
    option domain-name-servers 8.8.8.8;
    default-lease-time 600;
    max-lease-time 7200;
}
```

Figure 2.2: DHCP Configuration File

Description:

This figure shows the DHCP configuration, including the subnet (192.168.6.0/24), IP range (192.168.6.100 – 192.168.6.200), default gateway, and DNS server.

2.3 DHCP Service Initialization

```
[root@localhost ~]# systemctl start dhcpd
[root@localhost ~]# systemctl enable dhcpd
Created symlink from /etc/systemd/system/multi-user.target.wants/dhcpd.service to /usr/lib/systemd/system/dhcpd.service.
[root@localhost ~]# systemctl status dhcpd
■ dhcpd.service - DHCPv4 Server Daemon
   Loaded: loaded (/usr/lib/systemd/system/dhcpd.service; enabled; vendor preset: disabled)
   Active: active (running) since Thu 2026-04-02 22:24:43 IST; 33s ago
     Docs: man:dhcpd(8)
           man:dhcpd.conf(5)
   Main PID: 2604 (dhcpd)
    Status: "Dispatching packets..."
   CGroup: /system.slice/dhcpd.service
           └─2604 /usr/sbin/dhcpd -f -cf /etc/dhcp/dhcpd.conf -user dhcpd -group dhcpd --no-pid

Apr 02 22:24:43 localhost.localdomain dhcpd[2604]: No subnet declaration for enp0s8 (10.0.3.15).
Apr 02 22:24:43 localhost.localdomain dhcpd[2604]: ** Ignoring requests on enp0s8. If this is ...at
Apr 02 22:24:43 localhost.localdomain dhcpd[2604]: you want, please write a subnet declaration
Apr 02 22:24:43 localhost.localdomain dhcpd[2604]: in your dhcpd.conf file for the network s...nt
Apr 02 22:24:43 localhost.localdomain dhcpd[2604]: to which interface enp0s8 is attached. **
Apr 02 22:24:43 localhost.localdomain dhcpd[2604]: Listening on LPF/enp0s3/08:00:27:3a:ea:be/19...24
Apr 02 22:24:43 localhost.localdomain dhcpd[2604]: Sending on LPF/enp0s3/08:00:27:3a:ea:be/19...24
Apr 02 22:24:43 localhost.localdomain dhcpd[2604]: Sending on Socket/fallback/fallback-net
Apr 02 22:24:43 localhost.localdomain systemd[1]: Started DHCPv4 Server Daemon.
Hint: Some lines were ellipsized, use -l to show in full.
[root@localhost ~]#
```

Figure 2.3: DHCP Service Running Status

Description:

This output clearly illustrates the DHCP service running successfully on the CentOS server, confirming it is active and ready to assign IP addresses to clients.

2.4 Client Configuration for DHCP

```
GNU nano 6.2 /etc/netplan/01-netcfg.yaml
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: yes
```

Figure 2.4: Netplan Configuration for DHCP (Client Machine)

Description:

The Netplan Configuration file with `dhcp4: yes`, enabling the client to receive an IP address automatically from the DHCP server.

2.5 DHCP IP Assignment Verification

```
vimukthi@client1:~$ sudo cat /etc/netplan/*.yaml
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: yes
vimukthi@client1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:94:c5:18 brd ff:ff:ff:ff:ff:ff
    inet 192.168.6.100/24 metric 100 brd 192.168.6.255 scope global dynamic enp0s3
        valid_lft 416sec preferred_lft 416sec
    inet6 fe80::a00:27ff:fe94:c518/64 scope link
        valid_lft forever preferred_lft forever
vimukthi@client1:~$
```

Figure 2.5: DHCP IP Assignment on Client 1

Description:

This result confirms that the Ubuntu client 1 has received a dynamic IP address from the DHCP server within the configured range.

```
vimukthi@client2:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:13:92:6e brd ff:ff:ff:ff:ff:ff
    inet 192.168.6.176/24 brd 192.168.6.255 scope global dynamic enp0s3
        valid_lft 597sec preferred_lft 597sec
    inet6 fe80::a00:27ff:fe13:926e/64 scope link
        valid_lft forever preferred_lft forever
vimukthi@client2:~$
```

Figure 2.6: DHCP IP Assignment on Client 2

Description:

This figure illustrates that Ubuntu client 2 receives a dynamic IP address from the DHCP server, confirming that the DHCP service is functioning correctly.

2.6 Network Connectivity Verification

```
vimukthi@client1:~$ ping 192.168.6.1
PING 192.168.6.1 (192.168.6.1) 56(84) bytes of data.
64 bytes from 192.168.6.1: icmp_seq=1 ttl=64 time=1.50 ms
64 bytes from 192.168.6.1: icmp_seq=2 ttl=64 time=1.24 ms
64 bytes from 192.168.6.1: icmp_seq=3 ttl=64 time=0.963 ms
64 bytes from 192.168.6.1: icmp_seq=4 ttl=64 time=1.12 ms
^C
--- 192.168.6.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 0.963/1.205/1.495/0.194 ms
vimukthi@client1:~$
```

Figure 2.7: Connectivity Test from Client 1 to Server

Description:

This result confirms successful ping communication from Client 1 to the CentOS server, verifying network connectivity.

```
vimukthi@client2:~$ ping -c 4 192.168.6.1
PING 192.168.6.1 (192.168.6.1) 56(84) bytes of data.
64 bytes from 192.168.6.1: icmp_seq=1 ttl=64 time=1.38 ms
64 bytes from 192.168.6.1: icmp_seq=2 ttl=64 time=1.19 ms
64 bytes from 192.168.6.1: icmp_seq=3 ttl=64 time=1.14 ms
64 bytes from 192.168.6.1: icmp_seq=4 ttl=64 time=0.737 ms

--- 192.168.6.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.737/1.111/1.384/0.235 ms
vimukthi@client2:~$
```

Figure 2.8: Connectivity Test from Client 2 to Server

Description:

The above output demonstrates successful ping communication from Client 2 to the Server, confirming proper IP assignment and connectivity.

3. Part C – Zimbra Mail Server

3.1 Server Pre-Configuration

```
[root@localhost ~]# hostnamectl set-hostname mail.it06.local
[root@localhost ~]# hostname
mail.it06.local
[root@localhost ~]#
```

Figure 3.1: Setting System Hostname

Description:

This screenshot demonstrates the configuration of the system hostname using the *hostnamectl set-hostname* command. The hostname is set to mail.it06.local, which is required for proper identification of the Zimbra mail server within the network.

```
GNU nano 2.3.1 File: /etc/hosts
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
192.168.6.1 mail.it06.local mail_
```

Figure 3.2: Configuring /etc/hosts File

Description:

This screenshot illustrates the configuration of the */etc/hosts* file, where the server IP address 192.168.6.1 is mapped to the hostname mail.it06.local. This ensures proper local name resolution is required for Zimbra services.

```
GNU nano 2.3.1 File: /etc/hosts
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
#::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
192.168.6.1 mail.it06.local mail
```

Figure 3.3: Verifying /etc/hosts Configuration

Description:

This figure shows the verified and updated */etc/hosts* file after configuration. The correct mapping confirms that the hostname is properly linked with the server IP address.

```
GNU nano 2.3.1          File: /etc/selinux/config

# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#   enforcing - SELinux security policy is enforced.
#   permissive - SELinux prints warnings instead of enforcing.
#   disabled - No SELinux policy is loaded.
SELINUX=disabled_
# SELINUXTYPE= can take one of three two values:
#   targeted - Targeted processes are protected,
#   minimum - Modification of targeted policy. Only selected processes are protected.
#   mls - Multi Level Security protection.
SELINUXTYPE=targeted
```

Figure 3.4: Configuring SELinux Settings

Description:

This figure shows the SELinux configuration file `/etc/selinux/config`. SELinux is set to disabled to avoid conflicts during the installation and operation of the Zimbra mail server.

```
[root@mail ~]# setenforce 0
[root@mail ~]# getenforce
Permissive
[root@mail ~]# _
```

Figure 3.5: Disabling SELinux Temporarily

Description:

This figure shows the use of commands ***setenforce 0*** and ***getenforce*** to temporarily disable SELinux. The output confirms that the system is running in Permissive/Disabled mode.

```
mail login: root
Password:
Last login: Fri Apr  3 17:06:48 on tty1
[root@mail ~]# getenforce
Disabled
[root@mail ~]#
```

Figure 3.6: Verifying SELinux Status

Description:

This screenshot confirms that SELinux has been successfully disabled by displaying the status as Disabled, ensuring compatibility with Zimbra installation.

3.2 Downloading & Installing Required Dependencies

```
base                                     | 3.6 kB  00:00:00
extras                                 | 2.9 kB  00:00:00
updates                                | 2.9 kB  00:00:00
Loading mirror speeds from cached hostfile
Resolving Dependencies
--> Running transaction check
---> Package epel-release.noarch 0:7-11 will be installed
--> Finished Dependency Resolution

Dependencies Resolved

=====
Package                Arch          Version      Repository      Size
=====
Installing:
epel-release            noarch        7-11         extras          15 k
=====

Transaction Summary
=====
Install 1 Package

Total download size: 15 k
Installed size: 24 k
Downloading packages:
epel-release-7-11.noarch.rpm           | 15 kB  00:00:00
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  Installing : epel-release-7-11.noarch                                1/1
  Verifying  : epel-release-7-11.noarch                                1/1

Installed:
epel-release.noarch 0:7-11

Complete!
[root@mail ~]# _
```

Figure 3.7: Downloading & Installing EPEL Repository

Description:

This figure shows the installation of the EPEL repository using the yum package manager. This repository provides additional packages required for Zimbra installation.

```

=====
Installing:
net-tools          x86_64          2.0-0.25.20131004git.el7          base          306 k
nmap-ncat          x86_64          2:6.40-19.el7                    base          206 k
unzip              x86_64          6.0-24.el7_9                     updates       172 k
wget              x86_64          1.14-18.el7_6.1                  base          547 k
Updating:
gmp                x86_64          1:6.0.0-15.el7                   base          281 k
sudo              x86_64          1.8.23-10.el7_9.3                updates       844 k

Transaction Summary
=====
Install  4 Packages
Upgrade  2 Packages

Total download size: 2.3 M
Downloading packages:
Delta RPMs disabled because /usr/bin/applydeltarpm not installed.
(1/6): gmp-6.0.0-15.el7.x86_64.rpm          | 281 kB  00:00:00
(2/6): net-tools-2.0-0.25.20131004git.el7.x86_64.rpm | 306 kB  00:00:00
(3/6): nmap-ncat-6.40-19.el7.x86_64.rpm      | 206 kB  00:00:00
(4/6): wget-1.14-18.el7_6.1.x86_64.rpm      | 547 kB  00:00:00
(5/6): unzip-6.0-24.el7_9.x86_64.rpm        | 172 kB  00:00:00
(6/6): sudo-1.8.23-10.el7_9.3.x86_64.rpm    | 844 kB  00:00:00
-----
Total                                          1.3 MB/s | 2.3 MB  00:00:01
Running transaction check
Running transaction test

Transaction check error:
  package gmp-1:6.0.0-15.el7.x86_64 is already installed

Error Summary
-----

[root@mail ~]#

```

Figure 3.8: Downloading and Installing Other Required Packages

Description:

This screenshot displays the installation of necessary system packages such as perl, unzip, and other dependencies using the yum command. These packages are essential for Zimbra to function properly.

3.3 Zimbra Installation Process

```

[root@mail sf_Downloads]# cp zcs-8.8.11_GA_3737.RHEL7_64.20181207111719.tgz /opt
[root@mail sf_Downloads]# cd /opt
[root@mail opt]# ls
UBoxGuestAdditions-7.2.6  zcs-8.8.11_GA_3737.RHEL7_64.20181207111719.tgz  zimbra.tgz
[root@mail opt]# _

```

Figure 3.9: Copying Zimbra Installation Files to /opt Directory

Description:

This figure illustrates copying the Zimbra compressed installation file (.tgz) into the /opt directory and extracting it. This step prepares the system for running the Zimbra installation script.

```

86_64.rpm
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/packages/zimbra-common-mbox-native-lib-8.8.11.1521095672-1.r7.x86_64.rpm
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/packages/zimbra-mbox-admin-console-war-8.8.11.1540805051-1.r7.x86_64.rpm
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/packages/zimbra-mbox-conf-8.8.11.1539627833-1.r7.x86_64.rpm
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/packages/zimbra-mbox-service-8.8.11.1543567590-1.r7.x86_64.rpm
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/packages/zimbra-mbox-store-libs-8.8.11.1539971682-1.r7.x86_64.rpm
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/packages/zimbra-mbox-war-8.8.11.1543567590-1.r7.x86_64.rpm
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/packages/zimbra-mbox-webclient-war-8.8.11.1544083326-1.r7.x86_64.rpm
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/packages/zimbra-timezone-data-1.0.1+1538986928-1.r7.x86_64.rpm
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/util/
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/util/modules/
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/util/modules/getconfig.sh
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/util/modules/packages.sh
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/util/modules/postinstall.sh
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/util/addUser.sh
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/util/globals.sh
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/util/utlfunc.sh
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/README.txt
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/install.sh
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/readme_binary_en_US.txt
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/.BUILD_TYPE
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/.BUILD_NUM
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/.BUILD_PLATFORM
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/.BUILD_RELEASE_NO
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/.BUILD_RELEASE_CANDIDATE
zcs-8.8.11_GA_3737.RHEL7_64.20181207111719/.BUILD_TIME_STAMP
[root@mail opt]#
[root@mail opt]# cd zcs-8.8.11_GA_3737.RHEL7_64.20181207111719
[root@mail zcs-8.8.11_GA_3737.RHEL7_64.20181207111719]# ./install.sh_

```

Figure 3.10: Extracting and Accessing Zimbra Installation Directory

Description:

This figure displays the extracted Zimbra installation files and shows navigation into the directory. The install.sh script is identified and used to start the installation process.

```
-----
Do you agree with the terms of the software license agreement? [N] Y

Use Zimbra's package repository [Y]

Importing Zimbra GPG key

Configuring package repository

Checking for installable packages

Found zimbra-core (local)
Found zimbra-ldap (local)
Found zimbra-logger (local)
Found zimbra-mta (local)
Found zimbra-dnscache (local)
Found zimbra-snmp (local)
Found zimbra-store (local)
Found zimbra-apache (local)
Found zimbra-spell (local)
Found zimbra-memcached (repo)
Found zimbra-proxy (local)
Found zimbra-drive (repo)
Found zimbra-imapd (local)
Found zimbra-patch (repo)

Select the packages to install

Install zimbra-ldap [Y]
```

Figure 3.11: Starting Zimbra Installation and Accepting License Agreement

Description:

The above figure shows the initial Zimbra installation process, including accepting the software license agreement and preparing the package repository for installation.

```

Installing:
  zimbra-core
  zimbra-ldap
  zimbra-logger
  zimbra-mta
  zimbra-dnscache
  zimbra-snmp
  zimbra-store
  zimbra-apache
  zimbra-spell
  zimbra-memcached
  zimbra-proxy
  zimbra-drive
  zimbra-patch
  zimbra-chat

The system will be modified.  Continue? [N] Y_

```

Figure 3.12: Selecting Zimbra Packages for Installation

Description:

This figure illustrates the selection of Zimbra components, including LDAP, MTA, Store, Proxy, and other required components for a complete mail server setup.

```

zimbra-chat will be downloaded and installed (later).

Downloading packages (11):
  zimbra-core-components
  zimbra-ldap-components
  zimbra-mta-components
  zimbra-dnscache-components
  zimbra-snmp-components
  zimbra-store-components
  zimbra-jetty-distribution
  zimbra-apache-components
  zimbra-spell-components
  zimbra-memcached
  zimbra-proxy-components
  ...done

Removing /opt/zimbra
Removing zimbra crontab entry...done.
Cleaning up zimbra init scripts...done.
Cleaning up /etc/security/limits.conf...done.

Finished removing Zimbra Collaboration Server.

Installing repo packages (11):
  zimbra-core-components
  zimbra-ldap-components
  zimbra-mta-components
  zimbra-dnscache-components
  zimbra-snmp-components
  zimbra-store-components
  zimbra-jetty-distribution
  zimbra-apache-components
  zimbra-spell-components
  zimbra-memcached
  zimbra-proxy-components
  ...

```

Figure 3.13: Installing Zimbra Packages

Description:

This result confirms the installation progress of Zimbra components. All required services and modules are being installed into the system.

3.4 Zimbra Configuration

```
It is suggested that the domain name have an MX record configured in DNS
Change domain name? [Yes] Yes
Create domain: [mail.it06.local] it06.local_
```

Figure 3.14: Domain Name Configuration During Installation

Description:

This figure shows the configuration of the mail domain mail.it06.local during installation. This domain is used for all email accounts created in the system.

```
Store configuration

 1) Status: Enabled
 2) Create Admin User: yes
 3) Admin user to create: admin@it06.local
** 4) Admin Password UNSET
 5) Anti-virus quarantine user: virus-quarantine.5tfwtsjyu@it06.local
 6) Enable automated spam training: yes
 7) Spam training user: spam.zv3uxleqq@it06.local
 8) Non-spam(Ham) training user: ham.ffesdjkjb@it06.local
 9) SMTP host: mail.it06.local
10) Web server HTTP port: 8080
11) Web server HTTPS port: 8443
12) Web server mode: https
13) IMAP server port: 7143
14) IMAP server SSL port: 7993
15) POP server port: 7110
16) POP server SSL port: 7995
17) Use spell check server: yes
18) Spell server URL: http://mail.it06.local:7780/aspell.php
19) Enable version update checks: TRUE
20) Enable version update notifications: TRUE
21) Version update notification email: admin@it06.local
22) Version update source email: admin@it06.local
23) Install mailstore (service webapp): yes
24) Install UI (zimbra,zimbraAdmin webapps): yes

Select, or 'r' for previous menu [r] 4

Password for admin@it06.local (min 6 characters): [EBe3Jfssd] Uimukthi02004_
```

Figure 3.15: Zimbra Main Configuration Menu and Setting Admin Password

Description:

This figure shows the Zimbra configuration menu and sets the administrator password for the Zimbra system. This account is later used to access the Zimbra Admin Console.

```

10) Web server HTTP port:      8080
11) Web server HTTPS port:    8443
12) Web server mode:          https
13) IMAP server port:         7143
14) IMAP server SSL port:     7993
15) POP server port:          7110
16) POP server SSL port:     7995
17) Use spell check server:    yes
18) Spell server URL:         http://mail.it06.local:7780/aspell.php
19) Enable version update checks: TRUE
20) Enable version update notifications: TRUE
21) Version update notification email: admin@it06.local
22) Version update source email: admin@it06.local
23) Install mailstore (service webapp): yes
24) Install UI (zimbra,zimbraAdmin webapps): yes

Select, or 'r' for previous menu [r] r

Main menu

  1) Common Configuration:
  2) zimbra-ldap:          Enabled
  3) zimbra-logger:        Enabled
  4) zimbra-mta:           Enabled
  5) zimbra-dnscache:      Enabled
  6) zimbra-snmp:          Enabled
  7) zimbra-store:         Enabled
  8) zimbra-spell:         Enabled
  9) zimbra-proxy:         Enabled
10) Default Class of Service Configuration:
   s) Save config to file
   x) Expand menu
   q) Quit

*** CONFIGURATION COMPLETE - press 'a' to apply
Select from menu, or press 'a' to apply config (? - help) a
Save configuration data to a file? [Yes] Yes

```

Figure 3.16: Saving Configuration and Applying Settings

Description:

This figure demonstrates saving the configuration settings and applying them to the system before completing the installation process.

3.5 Zimbra Installation Completion

```
Setting spam training and Anti-virus quarantine accounts...done.
Initializing store sql database...done.
Setting zimbraSmtphostname for mail.it06.local...done.
Configuring SNMP...done.
Setting up syslog.conf...done.
Starting servers...done.
Installing common zimlets...
  com_zimbra_adminversioncheck...done.
  com_zimbra_attachcontacts...done.
  com_zimbra_attachmail...done.
  com_zimbra_bulkprovision...done.
  com_zimbra_cert_manager...done.
  com_zimbra_clientuploader...done.
  com_zimbra_date...done.
  com_zimbra_email...done.
  com_zimbra_mailarchive...done.
  com_zimbra_phone...done.
  com_zimbra_proxy_config...done.
  com_zimbra_srchhighlighter...done.
  com_zimbra_tooltip...done.
  com_zimbra_url...done.
  com_zimbra_viewmail...done.
  com_zimbra_webex...done.
  com_zimbra_ymemoticons...done.
  com_zextras_drive_open...done.
  com_zextras_chat_open...done.
Finished installing common zimlets.
Restarting mailboxd...done.
Creating galsync account for default domain...done.

You have the option of notifying Zimbra of your installation.
This helps us to track the uptake of the Zimbra Collaboration Server.
The only information that will be transmitted is:
  The VERSION of zcs installed (8.8.11_GA_3737_RHEL7_64)
  The ADMIN EMAIL ADDRESS created (admin@it06.local)

Notify Zimbra of your installation? [Yes] _
```

Figure 3.17: Ready for Installation

Description:

This figure shows that all configuration settings are applied, and the installation process begins.

```

com_zimbra_clientuploader...done.
com_zimbra_date...done.
com_zimbra_email...done.
com_zimbra_mailarchive...done.
com_zimbra_phone...done.
com_zimbra_proxy_config...done.
com_zimbra_srchhighlighter...done.
com_zimbra_tooltip...done.
com_zimbra_url...done.
com_zimbra_viewmail...done.
com_zimbra_webex...done.
com_zimbra_ymemoticons...done.
com_zextras_drive_open...done.
com_zextras_chat_open...done.
Finished installing common zimlets.
Restarting mailboxd...done.
Creating galsync account for default domain...done.

You have the option of notifying Zimbra of your installation.
This helps us to track the uptake of the Zimbra Collaboration Server.
The only information that will be transmitted is:
  The VERSION of zcs installed (8.8.11_GA_3737_RHEL7_64)
  The ADMIN EMAIL ADDRESS created (admin@it06.local)

Notify Zimbra of your installation? [Yes]
Notifying Zimbra of installation via http://www.zimbra.com/cgi-bin/notify.cgi?VER=8.8.11_GA_3737_RHE
L7_64&MAIL=admin@it06.local

ERROR: Notification failed
Checking if the NG started running...done.
Setting up zimbra crontab...done.

Moving /tmp/zmsetup.28268403-192051.log to /opt/zimbra/log

Configuration complete - press return to exit

```

Figure 3.18: Finalizing Zimbra Installation

Description: This figure shows the completion of Zimbra installation, including initialization of services and optional notification to Zimbra servers.

```

[zimbra@mail ~]# zmcontrol status
Host mail.it06.local

amavis           Running
antispam         Running
antivirus        Running
dnscache         Running
ldap             Running
logger           Running
mailbox          Running
memcached        Running
mta              Running
opendkim         Running
proxy            Running
service webapp   Running
snmp             Running
spell            Running
stats            Running
zimbra webapp    Running
zimbraAdmin webapp Running
zimlet webapp    Running
zmconfigd        Running
[zimbra@mail ~]# _

```

Figure 3.19: Zimbra Service Status Verification

Description:

This figure shows the output of **zmcontrol status**, confirming that all Zimbra services, such as mailbox, LDAP, MTA, and proxy are running successfully.

3.6 Testing Mail Functionality

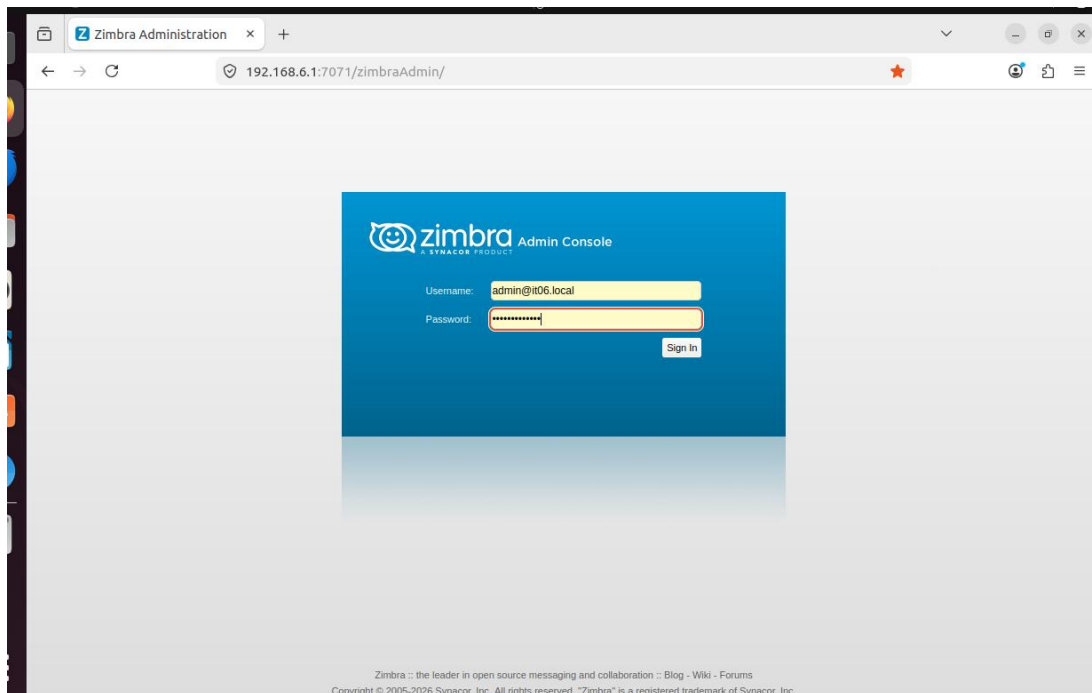


Figure 3.20: Zimbra Admin Console Login Page

Description:

This result confirms the Zimbra Admin Console login page accessed via the browser using the server's IP address and port 7071. The administrator enters credentials to manage the mail server.

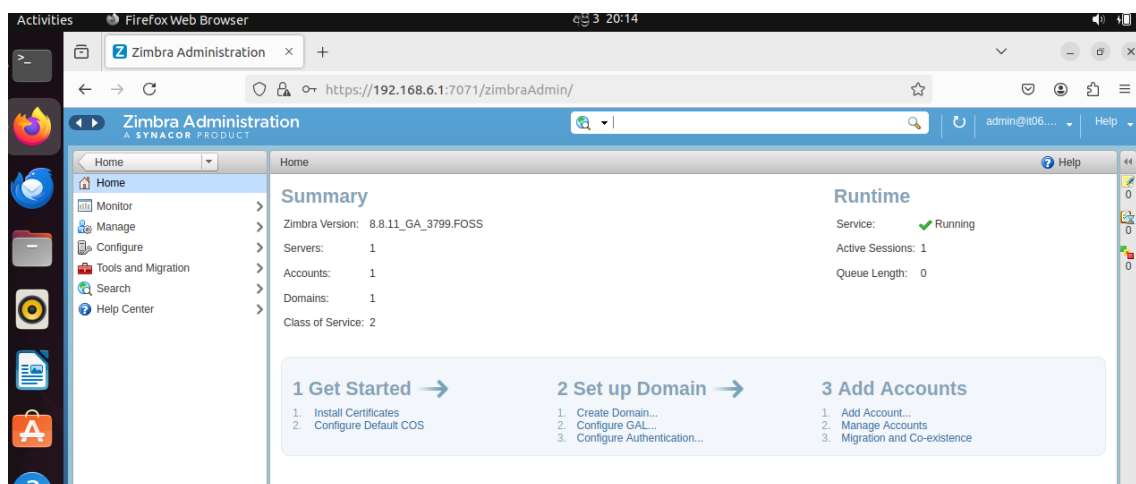


Figure 3.21: Zimbra Admin Dashboard

Description:

The figure displays the Zimbra Admin Console dashboard after successful login. It shows system status, active services, and options to manage domains, accounts, and configurations.

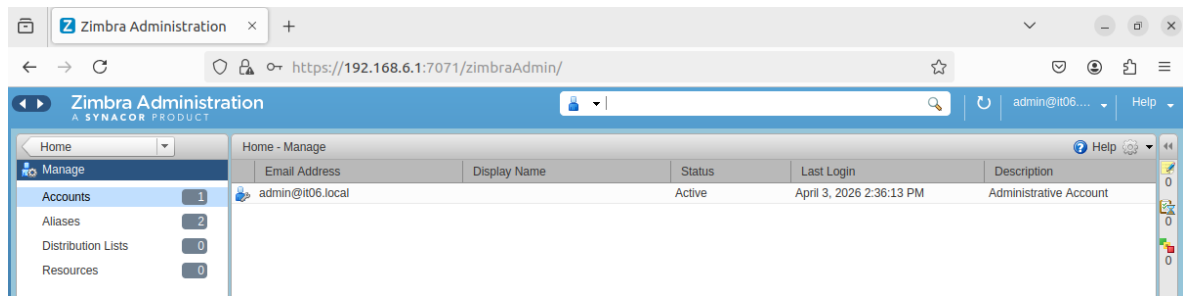


Figure 3.22: Viewing Existing Accounts in Admin Console

Description:

The figure illustrates the list of email accounts in the Zimbra Admin Console under the “Manage > Accounts” section. The default admin account is visible.

New Account

General Information

Account Name: user1 @ it06.local

First name: User

Middle initial:

Last name: One

Display name: User One ☒ auto

☐ Hide in GAL:

Account Setup

Status: Active

Buttons: Help, Cancel, Previous, Next, Finish

Figure 3.23: Creating a New Mail User Account

Description:

This figure illustrates the process of creating a new user account by entering details such as account name, first name, and display name within the configured domain.

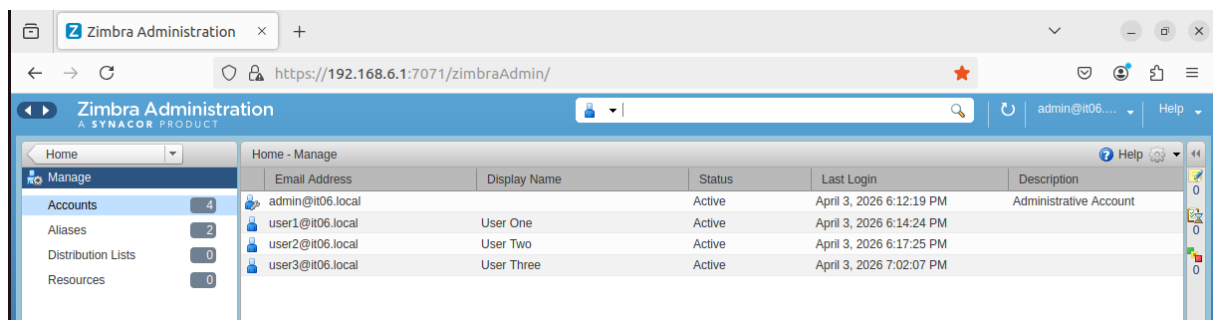


Figure 3.24: Multiple User Accounts Created Successfully

Description:

This figure illustrates the multiple user accounts successfully created in the system. Each account is active and ready for sending and receiving emails.

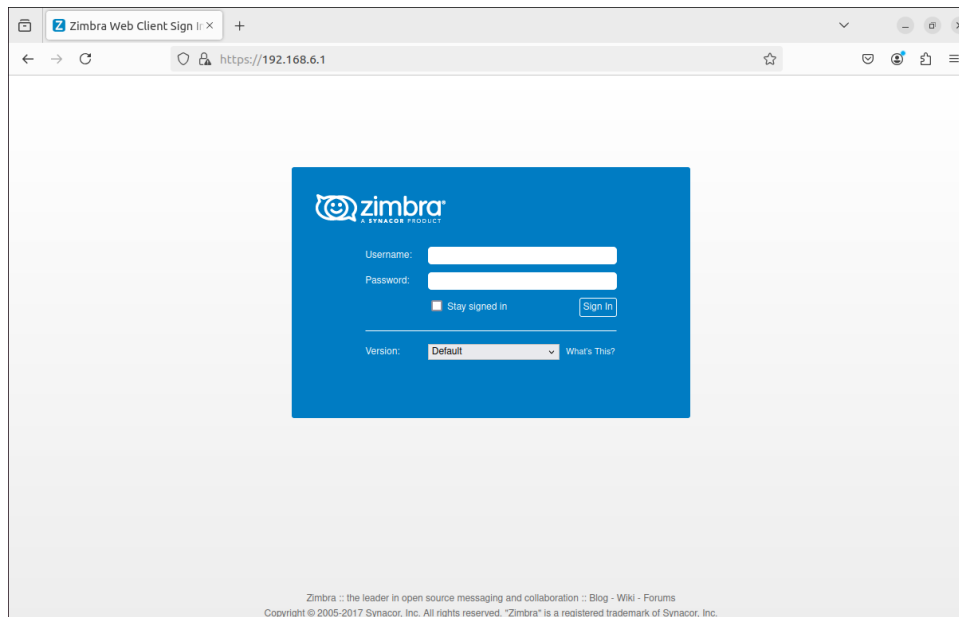


Figure 3.25: Zimbra Web Client Login Page

Description:

This figure shows the Zimbra Web Client login interface accessed via the server's IP address. Users log in using their email credentials to access mailbox services.

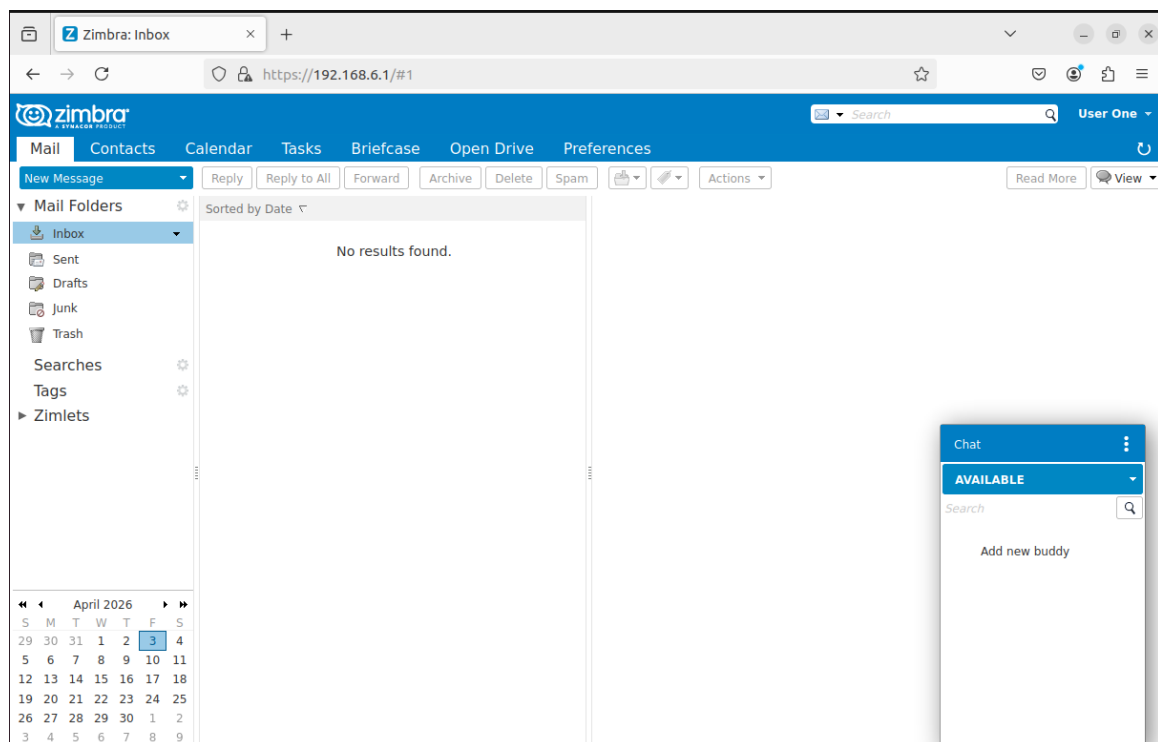


Figure 3.26: User Mailbox Interface (Inbox View of User One)

Description:

The screenshot demonstrates a user's mailbox interface after logging in. It includes folders such as the inbox and Drafts, as well as options to compose and manage emails.

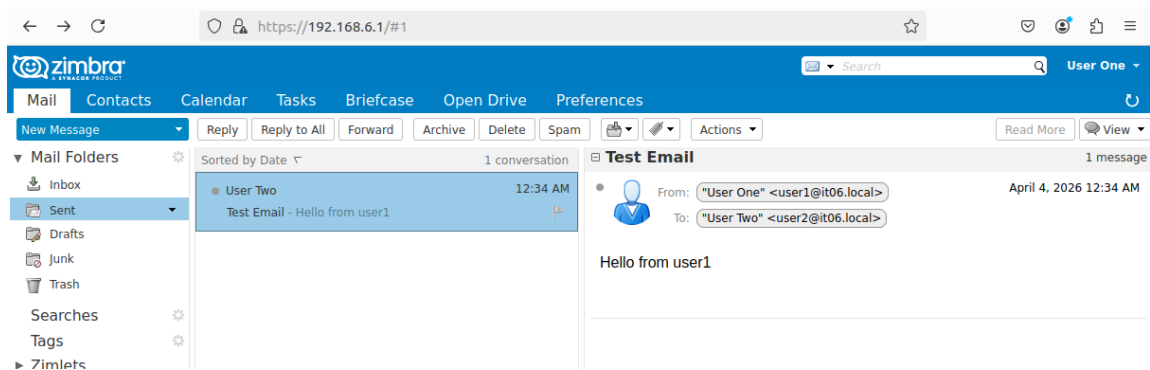


Figure 3.27: Email Sent from User One to User Two

Description:

This figure shows an email sent from user1@it06.local to user2@it06.local. The message appears in the Sent folder, confirming successful sending.

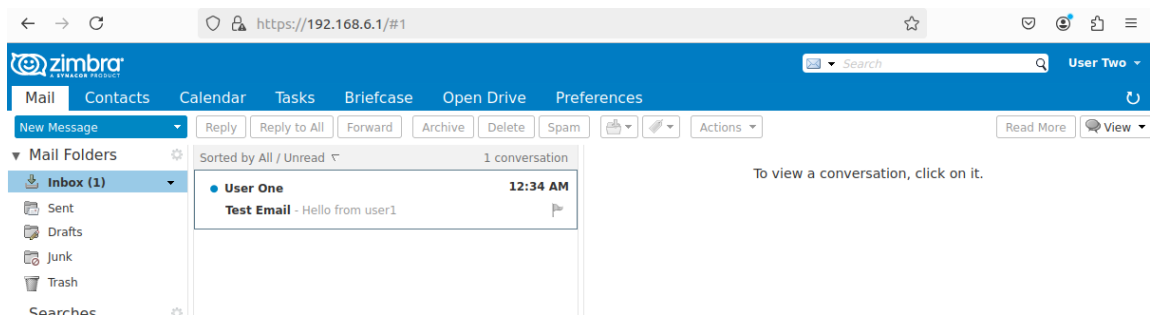


Figure 3.28: Email Received in User Two's Inbox

Description:

This result confirms that the received email is in User Two's inbox. It confirms successful mail delivery within the Zimbra mail server.

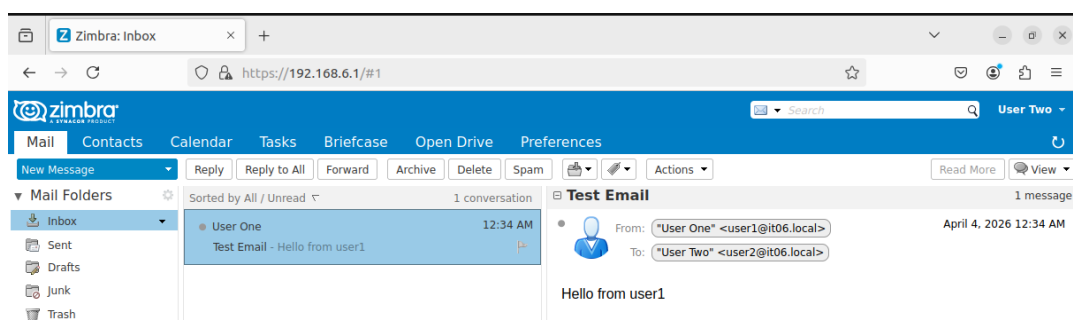


Figure 3.29: Viewing Received Email Content

Description:

This figure displays an open email message in the inbox, showing sender, recipient, subject, and message body content.

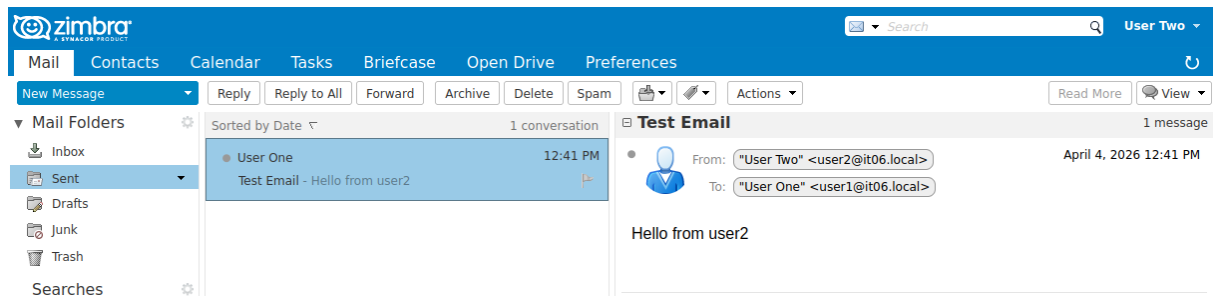


Figure 3.30: Reply Email from User Two to User One

Description:

This screenshot shows a reply email sent from User Two back to User One, demonstrating two-way communication between users.

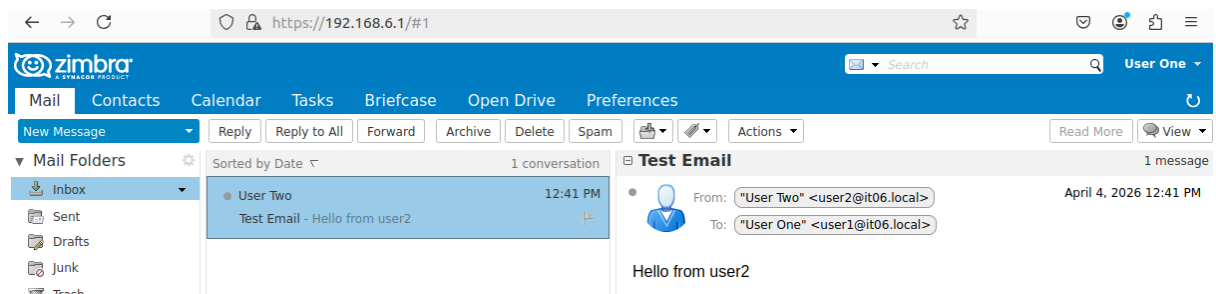


Figure 3.31: Reply Email Received in User One's Inbox

Description:

This figure shows the reply email received in User One's inbox, confirming that both sending and receiving functionalities are working correctly.

Conclusion:

The Zimbra mail server was successfully installed and configured within a virtualized network environment. User accounts were created, and email communication between users was successfully tested. The successful implementation demonstrates the ability to configure and manage a mail server within a virtual network environment, fulfilling all assignment requirements.

4. Part D – Samba Active Directory Domain Controller

Note:

Although the assignment specifies using a CentOS server, the Samba Active Directory Domain Controller was implemented on an Ubuntu Server due to compatibility limitations in CentOS 7. CentOS 7 provides older Samba versions that offer limited support for modern Active Directory features and require complex configurations. In contrast, Ubuntu provides updated Samba packages with improved Kerberos and DNS integration, enabling a more stable and efficient domain controller setup. Additionally, since the Zimbra Mail Server was already configured on the CentOS machine using the same domain environment, deploying Samba AD on a separate Ubuntu Server helped avoid potential conflicts and ensured stable operation of both services. The full qualified domain name (FQDN) mail.it06.local was used for Samba AD and Kerberos configurations. This ensured proper DNS resolution and successful domain authentication within the network.

Therefore, Ubuntu Server was selected to ensure the successful implementation and proper functioning of domain services.

4.1 Initial Setup and Kerberos Configuration

```
root@vimu:~# sudo apt update
Hit:1 http://lk.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://lk.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:4 http://lk.archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
120 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@vimu:~#
```

Figure 4.1: Updating Ubuntu Server Packages

Description:

This figure confirms the execution of the *sudo apt update* command on the Ubuntu Server. This step ensures that all package lists are up to date before installing and configuring the required services, such as Samba Active Directory Domain Controller and Kerberos. Keeping the system up to date helps avoid compatibility issues during configuration.

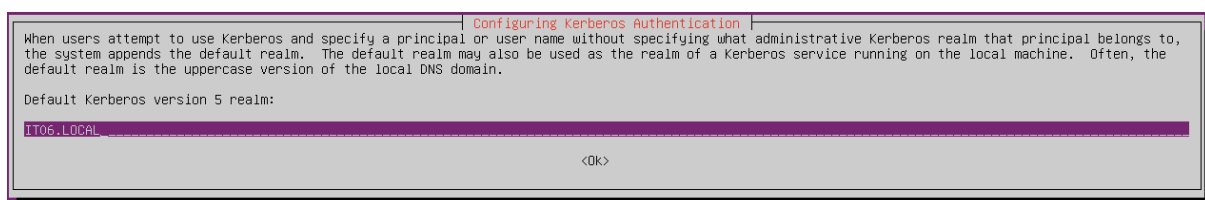


Figure 4.2: Configuring Default Kerberos Realm

Description:

This screenshot shows the configuration of the default Kerberos realm as IT06.LOCAL, which defines the authentication domain.

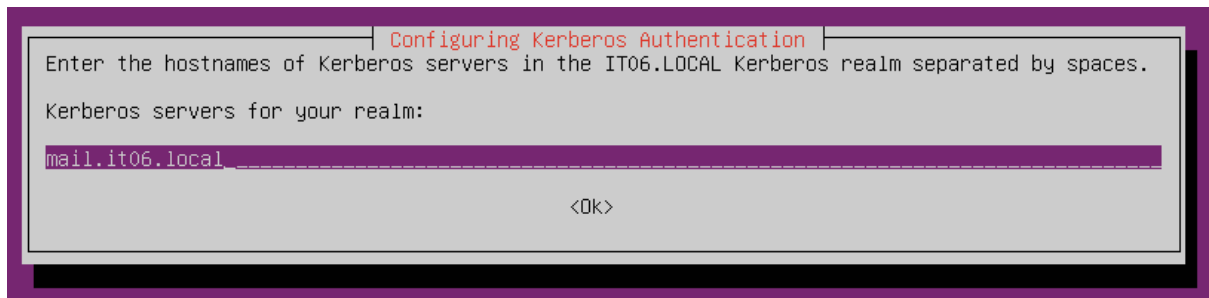


Figure 4.3: Specifying Kerberos Server Hostname

Description:

This figure illustrates the Kerberos server hostname set to mail.it06.local, enabling clients to locate the authentication server.

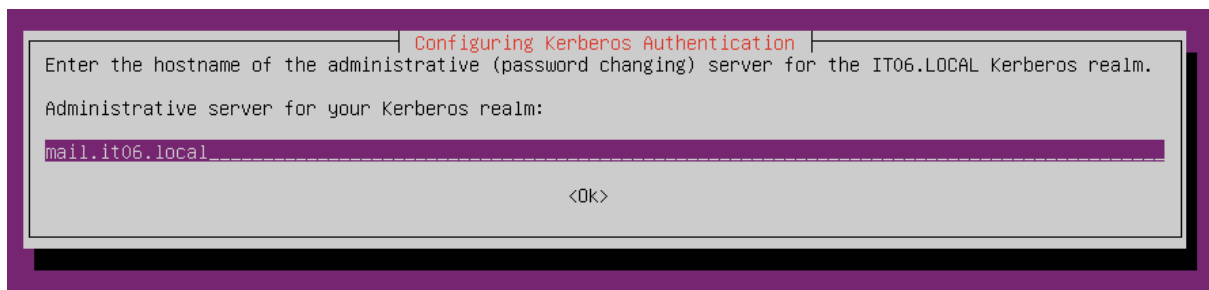


Figure 4.4: Configuring Kerberos Administrative Server

Description:

This screenshot illustrates the setup of the Kerberos administrative server, which manages authentication and password-related operations.

4.2 Domain Provisioning and Service Configuration

```
vimukthi@vimu:~$ sudo samba-tool domain provision
Realm: IT06.LOCAL
Domain [IT06]: IT06
Server Role (dc, member, standalone) [dc]: dc
DNS backend (SAMBA_INTERNAL, BIND9_FLATFILE, BIND9_DLZ, NONE) [SAMBA_INTERNAL]: SAMBA_INTERNAL
DNS forwarder IP address (write 'none' to disable forwarding) [127.0.0.53]: 8.8.8.8
Administrator password:
Retype password: _
```

Figure 4.5: Provisioning Samba Active Directory Domain

Description:

This figure shows the execution of the domain provisioning command, creating the IT06.LOCAL domain with the server configured as a Domain Controller.

```

INFO 2026-04-04 16:42:24,640 pid:5783 /usr/lib/python3/dist-packages/samba/provision/__init__.py #492: Server Role: active directory domain controller
INFO 2026-04-04 16:42:24,640 pid:5783 /usr/lib/python3/dist-packages/samba/provision/__init__.py #493: Hostname: vimu
INFO 2026-04-04 16:42:24,640 pid:5783 /usr/lib/python3/dist-packages/samba/provision/__init__.py #494: NetBIOS Domain: IT06
INFO 2026-04-04 16:42:24,647 pid:5783 /usr/lib/python3/dist-packages/samba/provision/__init__.py #495: DNS Domain: it06.local
INFO 2026-04-04 16:42:24,647 pid:5783 /usr/lib/python3/dist-packages/samba/provision/__init__.py #496: DOMAIN SID: S-1-5-21-514893744-2244278038-1034
347114
vimukthi@vimu:~$ _

```

Figure 4.6: Successful Samba Domain Provisioning Output

Description:

This screenshot displays the final output of the Samba domain provisioning process. It confirms that the Active Directory domain IT06.LOCAL has been created successfully. This indicates that the domain controller setup has been completed correctly.

```

• samba-ad-dc.service - Samba AD Daemon
  Loaded: loaded (/lib/systemd/system/samba-ad-dc.service; enabled; vendor preset: enabled)
  Active: active (running) since Sat 2026-04-04 16:45:11 +0530; 20s ago
    Docs: man:samba(8)
           man:samba(7)
           man:smb.conf(5)
  Main PID: 6534 (samba)
  Status: "samba: ready to serve connections..."
    Tasks: 59 (limit: 4598)
  Memory: 202.0M
    CPU: 4.399s
  CGroup: /system.slice/samba-ad-dc.service
          └─6534 "samba: root process"
              └─6540 "samba: tfork waiter process(6541)"
                  └─6541 "samba: task[s3fs] pre-fork master"
                      └─6542 "samba: tfork waiter process(6544)"
                          └─6543 "samba: tfork waiter process(6545)"
                              └─6544 "samba: task[rpc] pre-fork master"
                                  └─6545 /usr/sbin/smbd -D "--option=server role check:inhibit=yes" --foreground
                                      └─6546 "samba: tfork waiter process(6547)"
                                          └─6547 "samba: task[nbt] pre-fork master"
                                              └─6548 "samba: tfork waiter process(6549)"
                                                  └─6549 "samba: task[wrepl] pre-fork master"
                                                      └─6550 "samba: tfork waiter process(6551)"
                                                          └─6551 "samba: task[ldap] pre-fork master"
                                                              └─6552 "samba: tfork waiter process(6553)"
                                                                  └─6553 "samba: task[rpc] pre-forked worker(0)"
                                                                      └─6554 "samba: tfork waiter process(6556)"
                                                                          └─6555 "samba: tfork waiter process(6558)"
                                                                              └─6556 "samba: task[cldap] pre-fork master"
                                                                                  └─6557 "samba: tfork waiter process(6559)"
                                                                                      └─6558 "samba: task[rpc] pre-forked worker(1)"
                                                                                          └─6559 "samba: task[kdc] pre-fork master"
                                                                                              └─6560 "samba: tfork waiter process(6562)"
                                                                                                  └─6561 "samba: tfork waiter process(6563)"
                                                                                                      └─6562 "samba: task[drepl] pre-fork master"
                                                                                                          └─6563 "samba: task[rpc] pre-forked worker(2)"
                                                                                                              └─6564 "samba: tfork waiter process(6567)"
                                                                                                                  └─6565 "samba: tfork waiter process(6568)"
                                                                                                                      └─6566 "samba: tfork waiter process(6569)"
                                                                                                                          └─6567 "samba: task[winbindd] pre-fork master"
                                                                                                                              └─6568 "samba: task[rpc] pre-forked worker(3)"
                                                                                                                                  └─6569 "samba: task[kdc] pre-forked worker(0)"
                                                                                                                                      └─6571 "samba: tfork waiter process(6574)"
                                                                                                                                          └─6572 "samba: tfork waiter process(6576)"
                                                                                                                                              └─6573 /usr/sbin/smbd -D "--option=server role check:inhibit=yes" --foreground
                                                                                                                                                  └─6574 "samba: task[ntp_signd] pre-fork master"
                                                                                                                                  └─6575 /usr/sbin/smbd -D "--option=server role check:inhibit=yes" --foreground
                                                                                                                                      └─6576 "samba: task[kdc] pre-forked worker(1)"

```

Figure 4.7: Verifying Samba AD DC Service Status

Description:

This screenshot shows the status of the Samba Active Directory Domain Controller service using the `systemctl status samba-ad-dc` command. The service is shown as active (running), confirming that the domain controller is operational and ready to handle authentication and directory services within the network.

4.3 Organizational Units and User Management

```
vimukthi@vimu:~$ sudo samba-tool ou create "OU=Admins,DC=it06,DC=local"
Added ou "OU=Admins,DC=it06,DC=local"
vimukthi@vimu:~$ sudo samba-tool ou create "OU=Staff,DC=it06,DC=local"
Added ou "OU=Staff,DC=it06,DC=local"
vimukthi@vimu:~$ sudo samba-tool ou create "OU=Students,DC=it06,DC=local"
Added ou "OU=Students,DC=it06,DC=local"
vimukthi@vimu:~$
```

Figure 4.8: Creation of Organizational Units (OUs)

Descriptions:

This screenshot illustrates the creation of three Organizational Units: Admins, Staff, and Students using the *samba-tool ou create* command. These OUs are used to logically organize users within the domain, making administration and access control more structured and manageable.

```
vimukthi@vimu:~$ sudo samba-tool user create admin1 Vimukthi@2004
User 'admin1' added successfully
vimukthi@vimu:~$ sudo samba-tool user create admin2 Vimukthi@2004
User 'admin2' added successfully
vimukthi@vimu:~$ sudo samba-tool user create admin3 Vimukthi@2004
User 'admin3' added successfully
vimukthi@vimu:~$
vimukthi@vimu:~$ sudo samba-tool user create staff1 Vimukthi@2004
User 'staff1' added successfully
vimukthi@vimu:~$ sudo samba-tool user create staff2 Vimukthi@2004
User 'staff2' added successfully
vimukthi@vimu:~$ sudo samba-tool user create staff3 Vimukthi@2004
User 'staff3' added successfully
vimukthi@vimu:~$
vimukthi@vimu:~$ sudo samba-tool user create student1 Vimukthi@2004
User 'student1' added successfully
vimukthi@vimu:~$ sudo samba-tool user create student2 Vimukthi@2004
User 'student2' added successfully
vimukthi@vimu:~$ sudo samba-tool user create student3 Vimukthi@2004
User 'student3' added successfully
vimukthi@vimu:~$
```

Figure 4.9: Creation of Domain User Accounts

Descriptions:

This figure shows the creation of multiple user accounts under different categories, including Admins, Staff, and Students. Each user is successfully added to the domain using the *samba-tool user create* command, ensuring that the required user structure is properly implemented.

```

vimukthi@vimu:~$ sudo samba-tool user list
student3
staff2
Guest
staff3
Administrator
admin1
admin2
student2
admin3
staff1
student1
krbtgt
vimukthi@vimu:~$

```

Figure 4.10: Verification of Created Domain Users

Description:

This screenshot displays the output of the *samba-tool user list* command, which lists all users currently available in the domain. It confirms that all previously created user accounts have been successfully added and are recognized by Active Directory.

4.4 Domain Client Integration and Authentication

```

vimukthi@client2: ~
GNU nano 6.2 /etc/resolv.conf
nameserver 192.168.6.177
search it06.local

```

Figure 4.11: Configure DNS Settings on Client Machine

Description:

This shows the configuration of the */etc/resolv.conf* file on the client machine. The domain controller's IP address (192.168.6.177) is set as the nameserver, and the search domain is defined as it06.local. This configuration enables the client system to resolve domain names correctly and communicate with the Active Directory Domain Controller.

```

vimukthi@client2:~$ nslookup it06.local
Server:         192.168.6.177
Address:        192.168.6.177#53

Name:   it06.local
Address: 10.0.2.15
Name:   it06.local
Address: fd17:625c:f037:2:1e64:601a:1f57:caed
Name:   it06.local
Address: fd17:625c:f037:2:b4a3:b823:af5:2912
vimukthi@client2:~$

```

Figure 4.12: Verifying Domain Name Resolution Using nslookup

Description:

This output clearly illustrates the use of the *nslookup* command to verify that the domain it06.local resolves to the domain controller's IP address. This confirms that DNS configuration is properly configured and that the client can communicate with the domain controller.

```
vimukthi@client2:~$ realm discover it06.local
it06.local
  type: kerberos
  realm-name: IT06.LOCAL
  domain-name: it06.local
  configured: no
vimukthi@client2:~$
```

Figure 4.13: Discovering Domain Information Using realm Command

Description:

This screenshot displays the output of the realm discover command, which identifies the domain it06.local and its associated Kerberos configuration. This step verifies that the client system can detect the domain before attempting to join it.

```
vimukthi@client2:~$
vimukthi@client2:~$ sudo realm join it06.local -U administrator
[sudo] password for vimukthi:
Password for administrator:
vimukthi@client2:~$
```

Figure 4.14: Joining the Domain Using realm Command

Description:

This screenshot shows the execution of the realm join command using administrator credentials to join the client machine to the it06.local domain. This step integrates the client system into the Active Directory domain, allowing centralized authentication and management.

```
vimukthi@client2:~$ realm join it06.local -U administrator
realm: Already joined to this domain
vimukthi@client2:~$
```

Figure 4.15: Verification of Domain Join Status

Description:

This screenshot confirms that the client machine has already joined the domain. The message indicates that the system is successfully integrated into the domain.

```

vimukthi@client2:~$ realm list
it06.local
  type: kerberos
  realm-name: IT06.LOCAL
  domain-name: it06.local
  configured: kerberos-member
  server-software: active-directory
  client-software: sssd
  required-package: sssd-tools
  required-package: sssd
  required-package: libnss-sss
  required-package: libpam-sss
  required-package: adcli
  required-package: samba-common-bin
  login-formats: %U@it06.local
  login-policy: allow-realm-logins
vimukthi@client2:~$

```

Figure 4.16: Listing Domain configuration Details

Description:

This result confirms that the output of the *realm list* command, which provides detailed information about the joined domain, including the realm name, configuration type, and required packages. It confirms that the client is configured as a domain member.

```

vimukthi@client2:~$ id administrator@it06.local
uid=553000500(administrator@it06.local) gid=553000513(domain users@it06.local) groups=553000513(domain users@it06.local),553000519(enterprise admins@it06.local),553000518(schema admins@it06.local),553000520(group policy creator owners@it06.local),553000572(denied rodc password replication group@it06.local),553000512(domain admins@it06.local)
vimukthi@client2:~$

```

Figure 4.17: Verifying Domain User Identity

Description:

This figure illustrates the result of the *id* command for a domain user, showing user ID (UID), group ID (GID), and associated domain groups. This confirms that the domain user is recognized by the client system.

```

vimukthi@client2:~$ kinit administrator@IT06.LOCAL
Password for administrator@IT06.LOCAL:
Warning: Your password will expire in 41 days on 2026-05-16 16:42:21 +0530
vimukthi@client2:~$

```

Figure 4.18: Obtaining Kerberos Ticket Using kinit

Description:

This output clearly demonstrates the use of the *kinit* command to obtain a Kerberos ticket for the domain administrator. Successful authentication confirms that Kerberos is functioning correctly within the domain environment.

```
vimukthi@client2:~$ su - administrator@it06.local
Password:
Creating directory '/home/administrator@it06.local'.
administrator@it06.local@client2:~$
```

Figure 4.19: Logging in as Domain User

Description:

This screenshot illustrates switching to a domain user account using the **su** command. Successful login and home directory creation confirm that domain authentication is working properly.

```
vimukthi@client2:~$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: administrator@IT06.LOCAL

Valid starting     Expires            Service principal
2026-04-05 12:40:02 2026-04-05 22:40:02  krbtgt/IT06.LOCAL@IT06.LOCAL
        renew until 2026-04-06 12:39:58
vimukthi@client2:~$
```

Figure 4.20: Verifying Kerberos Ticket Using klist

Description:

This screenshot shows the output of the **klist** command, displaying the active Kerberos ticket for the authenticated user. This confirms that secure authentication has been established and is valid within the domain.

```
vimukthi@client1:~$ realm list
it06.local
  type: kerberos
  realm-name: IT06.LOCAL
  domain-name: it06.local
  configured: kerberos-member
  server-software: active-directory
  client-software: sssd
  required-package: sssd-tools
  required-package: sssd
  required-package: libnss-sss
  required-package: libpam-sss
  required-package: adcli
  required-package: samba-common-bin
  login-formats: %U@it06.local
  login-policy: allow-realm-logins
vimukthi@client1:~$
```

Figure 4.21: Domain Membership Verification on Client 1

Description:

This figure shows the output of the **realm list** command on Client 1, confirming that the system has successfully joined the it06.local domain.

```

vimukthi@client1:~$ id admin1@it06.local
uid=553001103(admin1@it06.local) gid=553000513(domain users@it06.local) groups=553000513(domain users@it06.local)
vimukthi@client1:~$ id staff1@it06.local
uid=553001106(staff1@it06.local) gid=553000513(domain users@it06.local) groups=553000513(domain users@it06.local)
vimukthi@client1:~$ id student1@it06.local
uid=553001109(student1@it06.local) gid=553000513(domain users@it06.local) groups=553000513(domain users@it06.local)
vimukthi@client1:~$

```

Figure 4.22: Verification of Multiple Domain Users on Client 1

Description:

This shows the output of the `id` command for multiple domain users, including admin, staff, and student accounts. The results confirm that the different user roles created in the Active Directory domain are successfully recognized and authenticated on the client system.

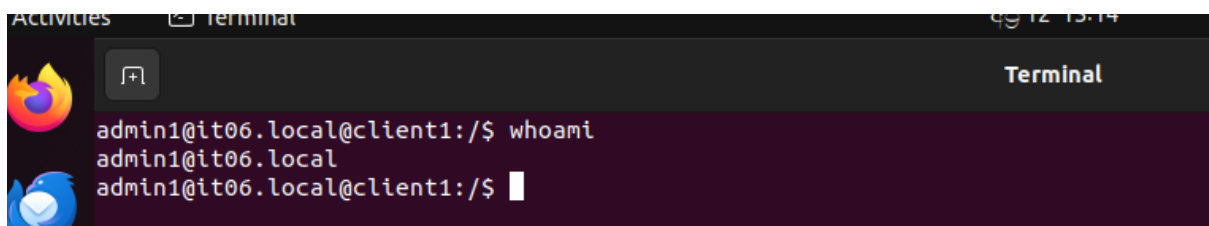
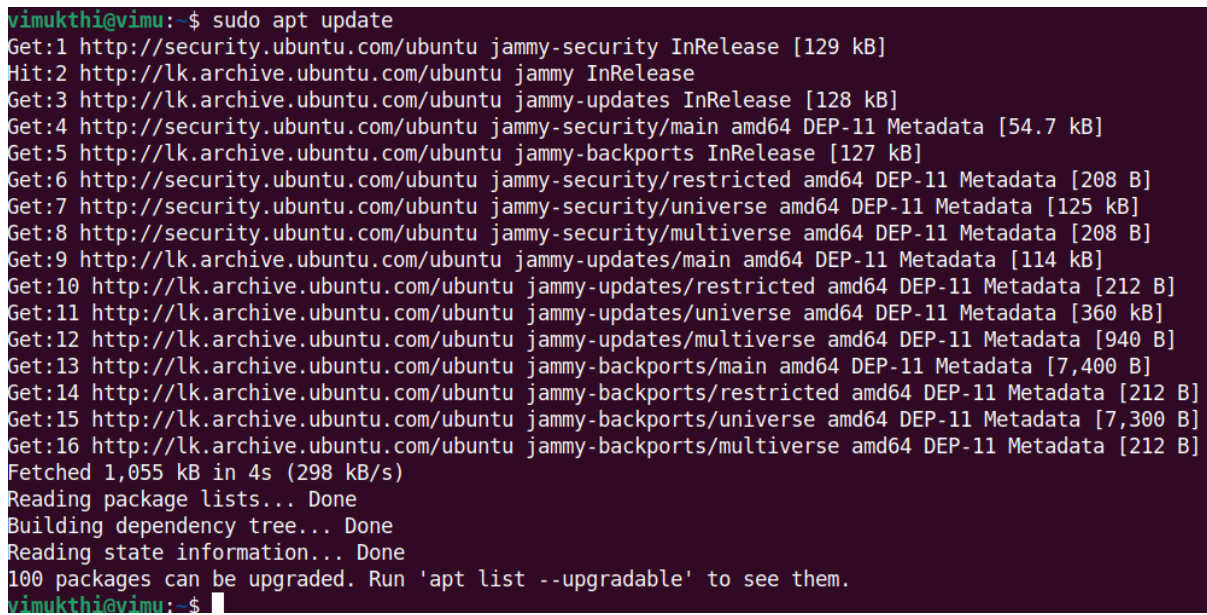


Figure 4.23: Successful Domain User GUI Login on Client 1

Description:

This output clearly shows a successful login to the Ubuntu client machine using a domain user account. The ***whoami*** command confirms that the user is authenticated through the Active Directory domain, demonstrating successful domain integration and authentication.

5. Part E – Web Server

A terminal window showing the output of the 'sudo apt update' command. The output lists various package repositories and their metadata sizes, such as 'jammy-security InRelease [129 kB]', 'jammy-updates InRelease [128 kB]', and 'jammy-backports InRelease [127 kB]'. It also shows the total size of the fetched data (1,055 kB) and the time taken (4s). The terminal ends with the prompt 'vimukthi@vimu:~\$' and a cursor.

```
vimukthi@vimu:~$ sudo apt update
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Hit:2 http://lk.archive.ubuntu.com/ubuntu jammy InRelease
Get:3 http://lk.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:4 http://security.ubuntu.com/ubuntu jammy-security/main amd64 DEP-11 Metadata [54.7 kB]
Get:5 http://lk.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:6 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 DEP-11 Metadata [208 B]
Get:7 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 DEP-11 Metadata [125 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 DEP-11 Metadata [208 B]
Get:9 http://lk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 DEP-11 Metadata [114 kB]
Get:10 http://lk.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 DEP-11 Metadata [212 B]
Get:11 http://lk.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 DEP-11 Metadata [360 kB]
Get:12 http://lk.archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 DEP-11 Metadata [940 B]
Get:13 http://lk.archive.ubuntu.com/ubuntu jammy-backports/main amd64 DEP-11 Metadata [7,400 B]
Get:14 http://lk.archive.ubuntu.com/ubuntu jammy-backports/restricted amd64 DEP-11 Metadata [212 B]
Get:15 http://lk.archive.ubuntu.com/ubuntu jammy-backports/universe amd64 DEP-11 Metadata [7,300 B]
Get:16 http://lk.archive.ubuntu.com/ubuntu jammy-backports/multiverse amd64 DEP-11 Metadata [212 B]
Fetched 1,055 kB in 4s (298 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
100 packages can be upgraded. Run 'apt list --upgradable' to see them.
vimukthi@vimu:~$
```

Figure 5.1: Updating Package Repository

Description:

This screenshot shows the execution of the ***sudo apt update*** command to update the system's package repository. This step ensures that the latest versions of required packages are available before installing the Apache web server.

A terminal window showing the output of the 'apt install apache2' command. The output indicates that the package lists are read, the dependency tree is built, and state information is read. It then states that 'apache2 is already the newest version (2.4.52-1ubuntu4.19)' and that '0 upgraded, 0 newly installed, 0 to remove and 93 not upgraded'. The terminal ends with the prompt 'root@vimu:~#'.

```
root@vimu:~# apt install apache2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apache2 is already the newest version (2.4.52-1ubuntu4.19).
0 upgraded, 0 newly installed, 0 to remove and 93 not upgraded.
root@vimu:~#
```

Figure 5.2: Apache Web Server Installation Verification

Description:

This figure shows the execution of the ***apt install apache2*** command on the Ubuntu server. The output shows that Apache is already installed and up to date, confirming that the web server package is successfully available on the system.

```

vimukthi@vimu:~$ sudo systemctl start apache2
[sudo] password for vimukthi:
vimukthi@vimu:~$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable apache2
vimukthi@vimu:~$
vimukthi@vimu:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2026-04-06 17:02:10 +0530; 2min 7s ago
     Docs: https://httpd.apache.org/docs/2.4/
  Main PID: 10762 (apache2)
    Tasks: 55 (limit: 4598)
   Memory: 5.2M
      CPU: 95ms
   CGroup: /system.slice/apache2.service
           └─10762 /usr/sbin/apache2 -k start
             └─10763 /usr/sbin/apache2 -k start
               └─10764 /usr/sbin/apache2 -k start

**** 06 17:02:10 vimu systemd[1]: Starting The Apache HTTP Server...
**** 06 17:02:10 vimu apachectl[10761]: AH00558: apache2: Could not reliably determine the server's fully qualified d
e
**** 06 17:02:10 vimu systemd[1]: Started The Apache HTTP Server.
lines 1-16/16 (END)

```

Figure 5.3: Starting and Verifying the Apache Web Server Service

Description:

This figure illustrates the Apache web server being started and enabled using *systemctl* command. The service status is displayed as Active (running), confirming that the web server is successfully running on the system.

```

GNU nano 6.2 /var/www/html/index.html
<!DOCTYPE html>
<html>
<head>
<title>NMA</title>
</head>
<body style="background-color:blue; color:white; text-align:center;">

<h1>HELLO NMA</h1>
</body>
</html>

```

Figure 5.4: Creating Web Page Content

Description:

This screenshot illustrates the creation and editing of the index.html file located in the */var/www/html/* directory. The HTML code defines a webpage with a blue background and displays the text “HELLO NMA” in a centered format.

```
vimukthi@vimu:~$ curl http://localhost
<!DOCTYPE html>
<html>
<head>
<title>NMA</title>
</head>
<body style="background-color:blue; color:white; text-align:center;">

<h1>HELLO NMA</h1>
</body>
</html>
vimukthi@vimu:~$
```

Figure 5.5: Testing Web Server Locally Using curl

Description:

This screenshot displays the output of the **curl** <http://localhost> command, which retrieves the HTML content of the hosted webpage. This confirms that the web server is correctly serving the created webpage locally.

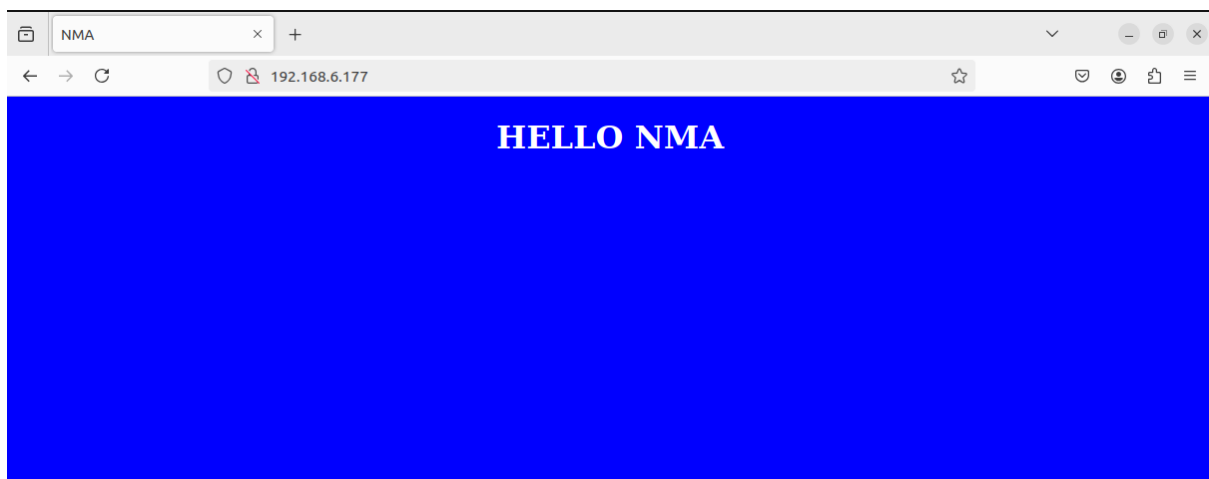


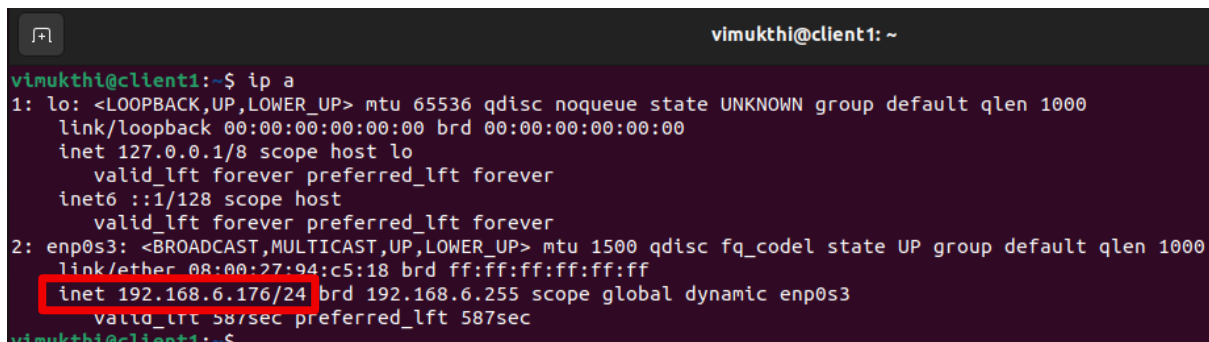
Figure 5.6: Accessing Web Page from Client Browser

Description:

The figure shows the webpage accessed through a web browser using the Ubuntu server's IP address. The page correctly displays "HELLO NMA" with a blue background, confirming that the web server is accessible from the client machine.

6. Final System Verification

The following verification steps confirm that all configured network services are functioning properly within the system.

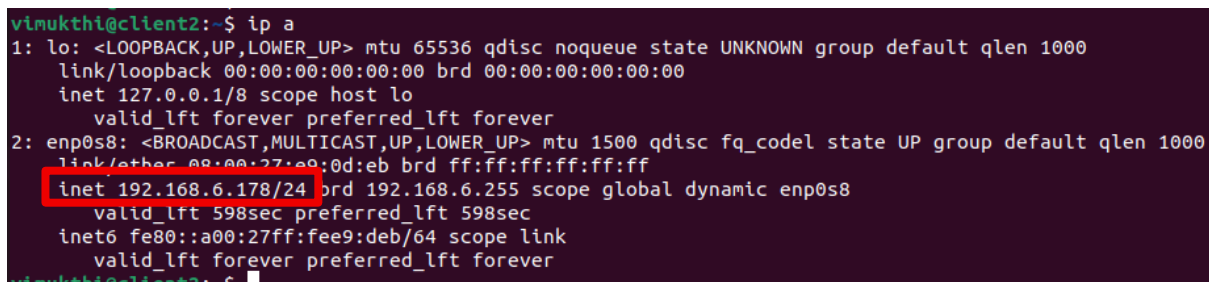


```
vimukthi@client1: ~  
vimukthi@client1:~$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host  
        valid_lft forever preferred_lft forever  
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:94:c5:18 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.6.176/24 brd 192.168.6.255 scope global dynamic enp0s3  
        valid_lft 587sec preferred_lft 587sec  
vimukthi@client1:~$
```

Figure 6.1: DHCP IP Assignment on Client 1

Description:

This result confirms the output of the *ip a* command on Ubuntu Client 1. The client has successfully received a dynamic IP address (192.168.6.176) from the DHCP server within the configured range, confirming that DHCP is functioning correctly.

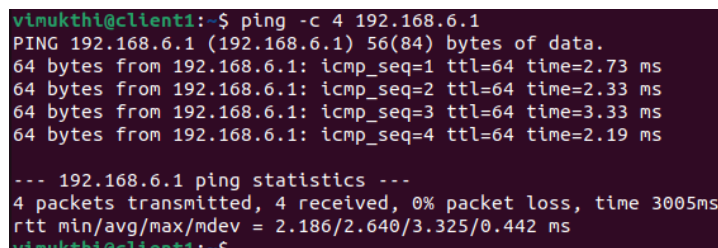


```
vimukthi@client2:~$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
2: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:94:0d:eb brd ff:ff:ff:ff:ff:ff  
    inet 192.168.6.178/24 brd 192.168.6.255 scope global dynamic enp0s8  
        valid_lft 598sec preferred_lft 598sec  
    inet6 fe80::a00:27ff:fee9:deb/64 scope link  
        valid_lft forever preferred_lft forever  
vimukthi@client2:~$
```

Figure 6.2: DHCP IP Assignment on Client 2

Description:

This screenshot shows the output of the *ip a* command on Ubuntu Client 2. The assigned IP address (192.168.6.178) confirms that the DHCP server is correctly allocating dynamic IP addresses to client machines.



```
vimukthi@client1:~$ ping -c 4 192.168.6.1  
PING 192.168.6.1 (192.168.6.1) 56(84) bytes of data:  
64 bytes from 192.168.6.1: icmp_seq=1 ttl=64 time=2.73 ms  
64 bytes from 192.168.6.1: icmp_seq=2 ttl=64 time=2.33 ms  
64 bytes from 192.168.6.1: icmp_seq=3 ttl=64 time=3.33 ms  
64 bytes from 192.168.6.1: icmp_seq=4 ttl=64 time=2.19 ms  
  
--- 192.168.6.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3005ms  
rtt min/avg/max/mdev = 2.186/2.640/3.325/0.442 ms  
vimukthi@client1:~$
```

Figure 6.3: Connectivity Test from Client 1 to Server

Description:

The above output illustrates the result of the *ping -c 4 192.168.6.1* command executed from client 1. All packets are successfully transmitted and received with 0% packet loss, confirming proper network connectivity.

```

vimukthi@client2:~$ ping -c 4 192.168.6.1
PING 192.168.6.1 (192.168.6.1) 56(84) bytes of data.
64 bytes from 192.168.6.1: icmp_seq=1 ttl=64 time=16.1 ms
64 bytes from 192.168.6.1: icmp_seq=2 ttl=64 time=3.24 ms
64 bytes from 192.168.6.1: icmp_seq=3 ttl=64 time=1.06 ms
64 bytes from 192.168.6.1: icmp_seq=4 ttl=64 time=2.11 ms

--- 192.168.6.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 1.061/5.618/16.072/6.083 ms
vimukthi@client2:~$

```

Figure 6.4: Connectivity Test from Client 2 to Server

Description:

This output demonstrates the result of the *ping -c 4 192.168.6.1* command executed from Client 2. The successful responses confirm that the client can communicate with the server over the network.

```

[root@mail vsiriwardanal# ping -c 4 192.168.6.176
PING 192.168.6.176 (192.168.6.176) 56(84) bytes of data.
64 bytes from 192.168.6.176: icmp_seq=1 ttl=64 time=4.62 ms
64 bytes from 192.168.6.176: icmp_seq=2 ttl=64 time=1.64 ms
64 bytes from 192.168.6.176: icmp_seq=3 ttl=64 time=1.59 ms
64 bytes from 192.168.6.176: icmp_seq=4 ttl=64 time=0.949 ms

--- 192.168.6.176 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3014ms
rtt min/avg/max/mdev = 0.949/2.203/4.625/1.424 ms
[root@mail vsiriwardanal#
[root@mail vsiriwardanal#
[root@mail vsiriwardanal# ping -c 4 192.168.6.178
PING 192.168.6.178 (192.168.6.178) 56(84) bytes of data.
64 bytes from 192.168.6.178: icmp_seq=1 ttl=64 time=2.18 ms
64 bytes from 192.168.6.178: icmp_seq=2 ttl=64 time=1.56 ms
64 bytes from 192.168.6.178: icmp_seq=3 ttl=64 time=4.28 ms
64 bytes from 192.168.6.178: icmp_seq=4 ttl=64 time=2.48 ms

--- 192.168.6.178 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3038ms
rtt min/avg/max/mdev = 1.562/2.629/4.288/1.015 ms
[root@mail vsiriwardanal#

```

Figure 6.5: Connectivity Test from Server to Clients

Description: This output confirms the result of the ping command executed from the server to both client machines (192.168.6.176 and 192.168.6.178). Successful responses with 0% packet loss confirm bidirectional network communication.

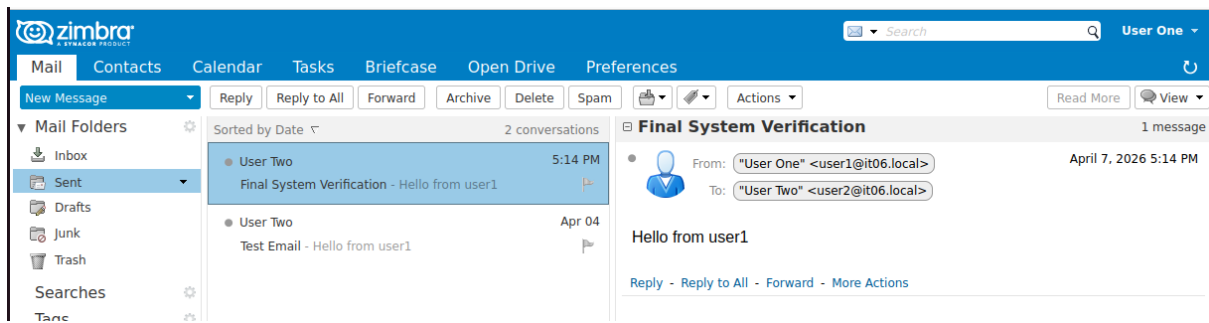


Figure 6.6: Email Sent from User One

Description:

This figure illustrates the Zimbra web client interface, where User One has successfully sent an email to User Two. The message appears in the Sent folder, confirming successful email transmission.

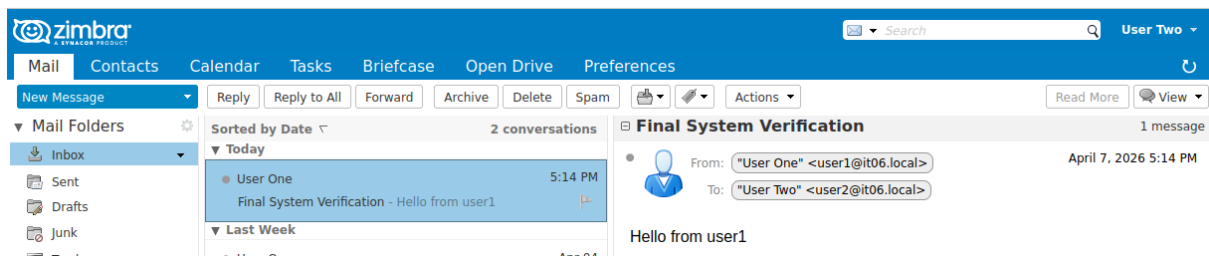


Figure 6.7: Email Received by User Two

Description:

The above output confirms that the Zimbra web client interface of User Two, where the email sent by User One is received in the inbox. This confirms successful email delivery within the mail server.

```
vimukthi@client2:~$ realm list
it06.local
  type: kerberos
  realm-name: IT06.LOCAL
  domain-name: it06.local
  configured: kerberos-member
  server-software: active-directory
  client-software: sssd
  required-package: sssd-tools
  required-package: sssd
  required-package: libnss-sss
  required-package: libpam-sss
  required-package: adcli
  required-package: samba-common-bin
  login-formats: %U
  login-policy: allow-realm-logins
vimukthi@client2:~$
```

Figure 6.8: Domain Discovery Using realm Command

Description:

This output confirms the output of the `realm list` command on the client machine. It confirms that the system is successfully joined to the domain (`it06.local`) and configured with Kerberos and SSSD for authentication.

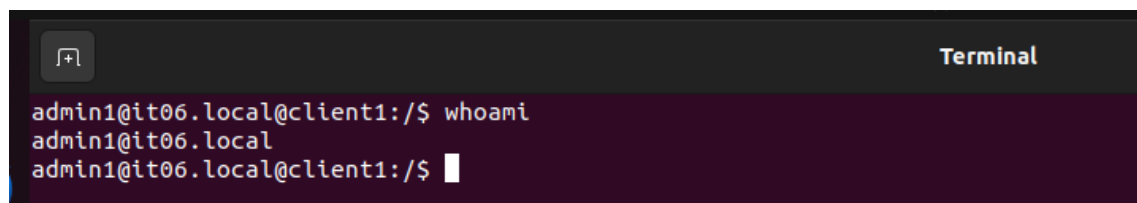
```
vimukthi@client2:~$ id student1@it06.local
uid=553001109(student1) gid=553000513(domain users) groups=553000513(domain users)
vimukthi@client2:~$
vimukthi@client2:~$ kinit student1@IT06.LOCAL
Password for student1@IT06.LOCAL:
Warning: Your password will expire in 38 days on 2026-05-16 16:49:04 +0530
vimukthi@client2:~$
vimukthi@client2:~$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: student1@IT06.LOCAL

Valid starting    Expires          Service principal
2026-04-07 17:50:55  2026-04-08 03:50:55  krbtgt/IT06.LOCAL@IT06.LOCAL
        renew until 2026-04-08 17:50:50
vimukthi@client2:~$
```

Figure 6.9: Domain User Authentication Verification

Description:

This screenshot demonstrates successful domain authentication. The `id` command confirms that the domain user is recognized, while the ***kinit*** and ***klist*** commands show that a valid Kerberos ticket has been obtained, confirming proper authentication.

A terminal window titled "Terminal" with a dark background. It shows a user logging in as "admin1@it06.local@client1". The prompt is "admin1@it06.local@client1:/\$". The user enters the command "whoami", and the output is "admin1@it06.local". The prompt returns to "admin1@it06.local@client1:/\$".

```
admin1@it06.local@client1:/$ whoami
admin1@it06.local
admin1@it06.local@client1:/$
```

Figure 6.10: Successful Domain User GUI Login on Client

Description:

This screenshot shows a successful login to the Ubuntu client machine using a domain user account. The ***whoami*** command confirms that the user is authenticated through the Active Directory domain, demonstrating successful domain integration and authentication.

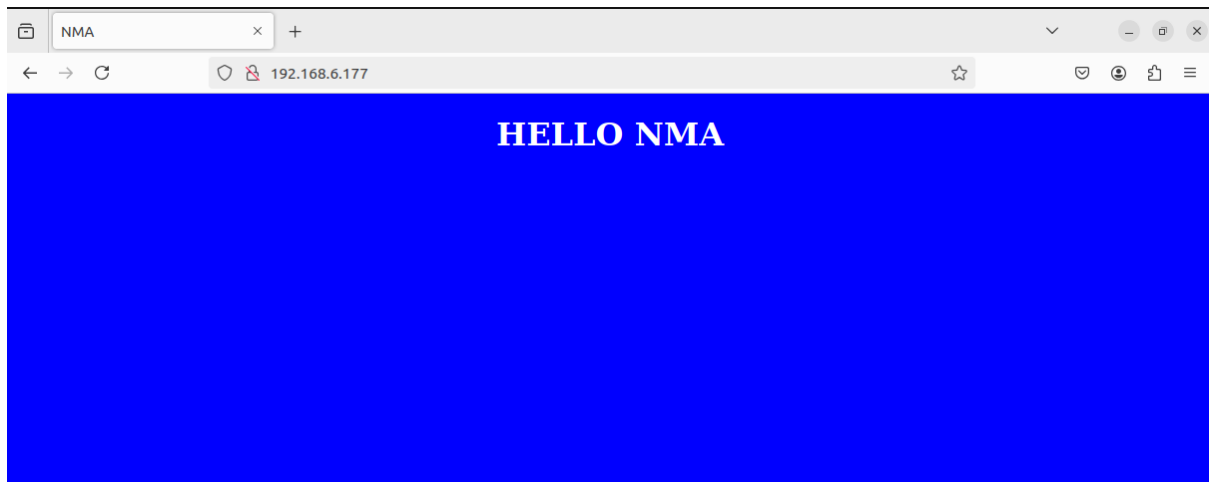


Figure 6.11: Accessing the Web Server from the Client Browser

Description:

This figure illustrates the hosted webpage accessed from the Ubuntu client using the Ubuntu server's IP address (192.168.6.177). The page displays "HELLO NMA" with a blue background, confirming that the web server is functioning correctly and accessible over the network.

- All services, including DHCP, Zimbra Mail Server, Samba Active Directory, and Web Server, were successfully verified and are functioning as expected. This demonstrates the successful integration of multiple network services within a virtualized environment, fulfilling all assignment requirements.